



科学研究的人工智能方法学

复旦大学人工智能创新与产业研究院研究员
徐盈辉

未来25年，这125个问题将让科学家“大伤脑筋”

这125个问题，其中涉及生命科学的问题占46%，关系宇宙和地球的问题占16%，与物质科学相关的问题占14%以上，认知科学问题占9%。其余问题分别涉及数学与计算机科学、政治与经济、能源、环境和人口等。

一场正在发生的科研范式变革



《 Science 》刊登麻省理工
AI药物逆合成设计论文，AI合
成了**15个**药物分子

2019.3

利物浦大学使用AI发现**4种**可
传导锂的新固态材料，可提
供更长续航能力

2021.5

Deepmind宣布开放包括多
个动物、植物的**2亿**个蛋白质
结构的预测结果

2022.7

Geometric deep learning: going beyond Euclidean data

Michael M. Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, Pierre Vandergheynst

<https://arxiv.org/pdf/1611.08097.pdf>

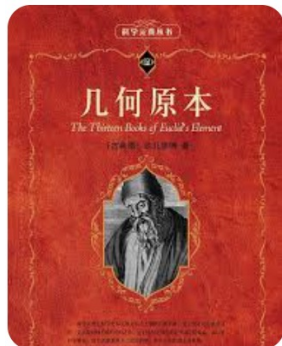


CS224W: Machine Learning with Graphs

Stanford / Fall 2024

<https://web.stanford.edu/class/cs224w/>

几何学的科学史



Euclid (~300BC)

几何学之父

欧式几何



N. Lobachevsky (1792-1856)



J. Bolyai (1802-1860)



C. F. Gauss (1777-1855)

罗氏几何



B. Riemann (1826-1866)

黎曼几何



F. Klein (1849-1925)

克莱因纲领

几何学+群论



E. Noether (1883-1935)

几何学+拓扑学

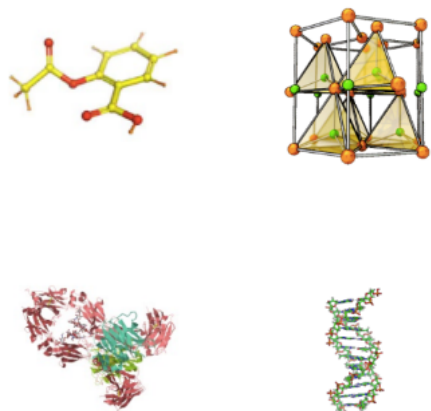


数据形式复杂性

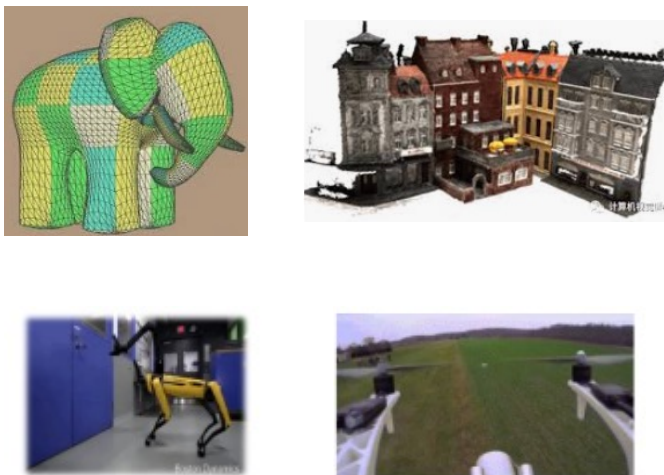
物理空间的数据和任务非常复杂，从微观的分子空间到宏观的具身空间，不仅研究尺度发生巨大变化，数据结构也没有统一的形态、唯一的形式，并且变量间的关联方式也是多样的

图数据无处不在，图是描述关联数据的通用语言

微观分子空间

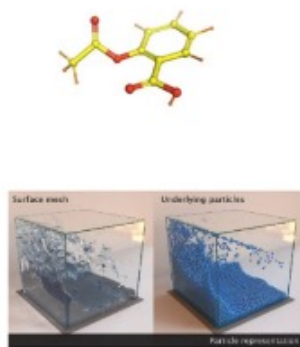


宏观具身空间

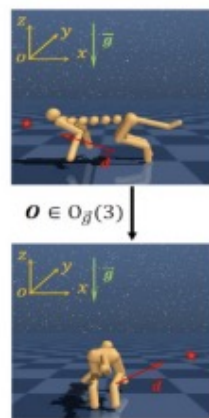


数据结构复杂

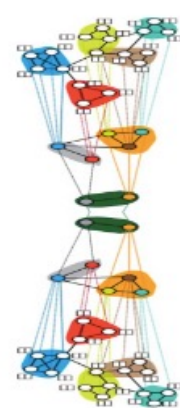
形态多样性



形式多样性



关联的复杂性





数据形式复杂性

几何深度学习是一种在提取特征时，具有保持对象特征不变特性的深度神经网络。这种变换在一些群上进行，如 2D 和 3D 欧氏空间中的平移群、旋转群、庞加莱群、洛伦兹群等。这些都会对应到所谓的“对称性 (Symmetry)”，这种对称性在物理上和生物上有很多应用。**能够把这些不变的性质和物理中的运动 and 对称性相结合是深度学习在各个科学领域得到成功的关键点。**

寻找一种通用化的数据表示，并以此为基础设计模型，其核心是从对称性或者不变性出发，将几何性质嵌入到机器学习流程当中，更好地刻画数据的内在结构和变化规律，提高算法的表示、泛化、通用能力。

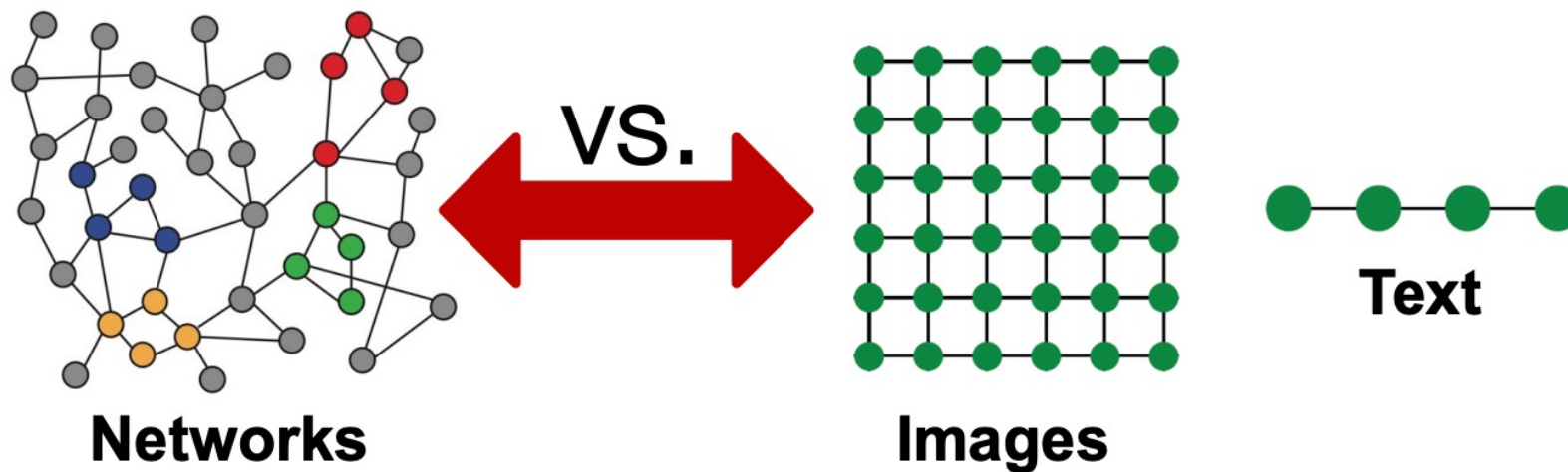


Michael Bronstein

牛津大学计算机科学教授，几何深度学习 (geometric deep learning) 概念的提出者

Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges,
<https://arxiv.org/abs/2104.13478>

图 @ 几何深度学习

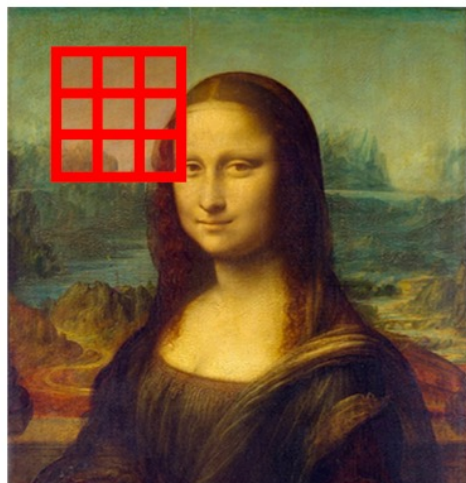


图数据的复杂性体现在其不规则的拓扑结构和无序的节点关系，这使得传统的基于规则网格的深度学习模型难以直接应用在图数据上

图结构的复杂拓扑

无固定的节点排序或参考点

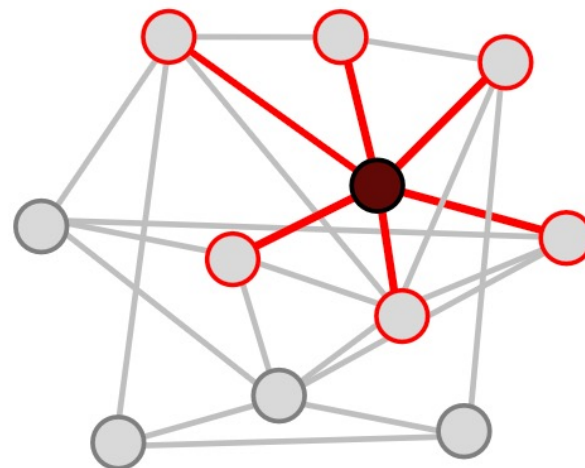
Images



- Constant number of neighbors
- Fixed ordering of neighbors
- Shift invariance

Convolution?

Graphs

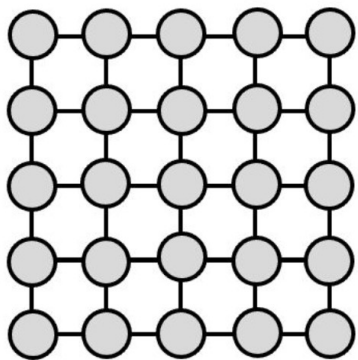


- Different number of neighbors
- No ordering of neighbors
- Permutation invariance

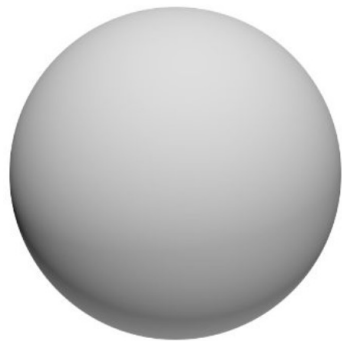
Pooling?

Efficient computation?

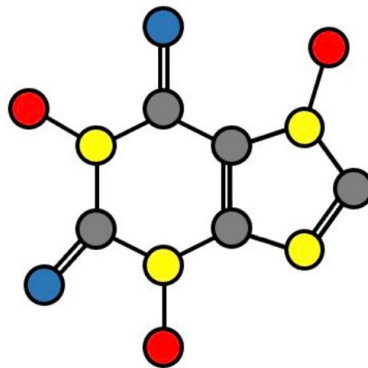
“5G” @ 几何深度学习



Grids



Groups



Graphs



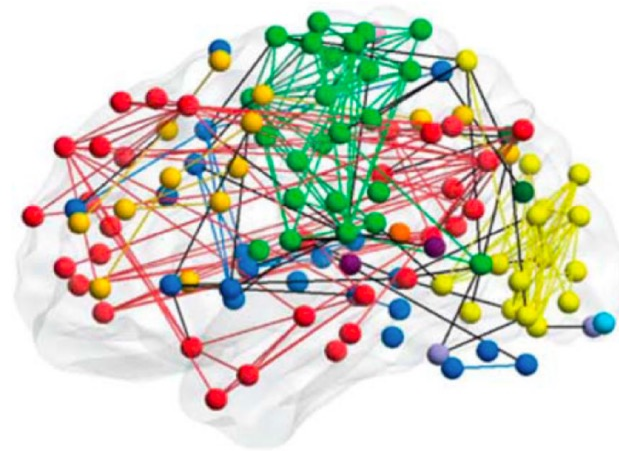
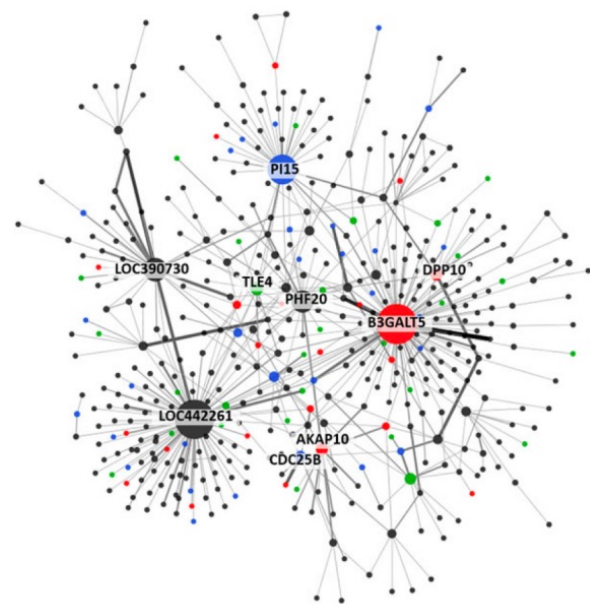
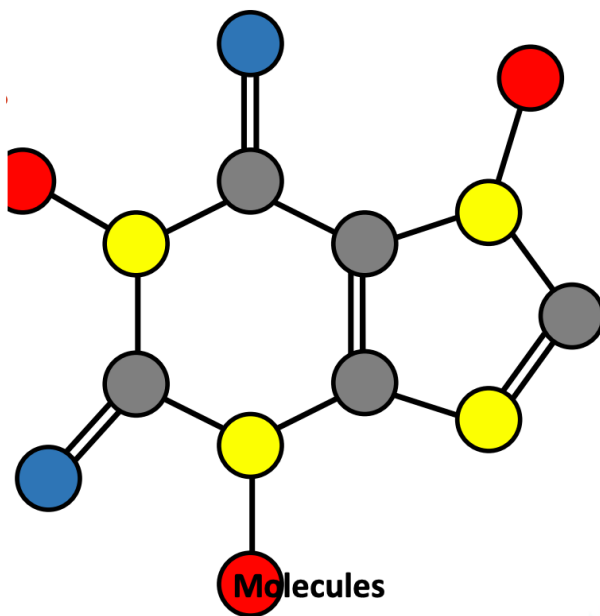
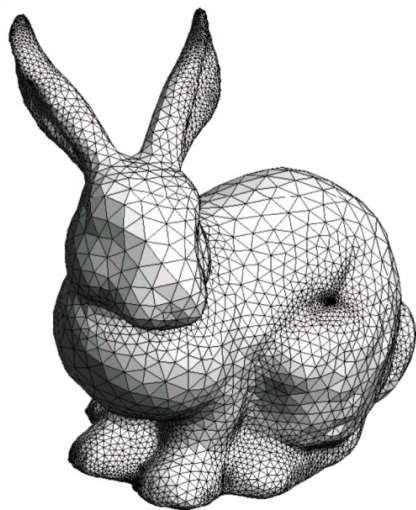
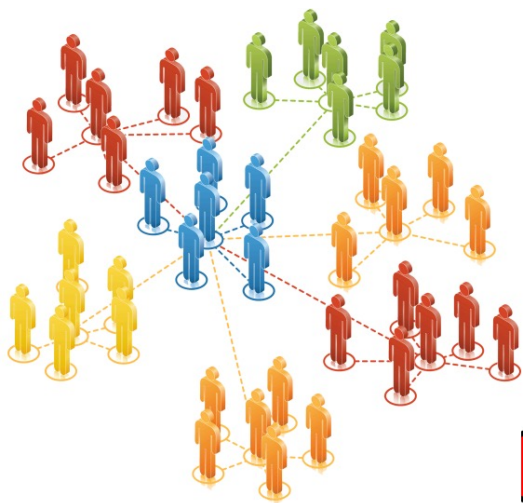
**Geodesics &
Gauges**

Grids (网格) :
通常表示规则的
网格结构, 比如
在图像处理中常
用的像素网格。

Groups (群)
表示对称性或对称
群, 通常用于建模
物体的对称特性,
帮助减少模型的计
算复杂度

Graphs (图)
表示由节点和边
组成的结构, 适
用于建模不规则
的关系, 比如社
交网络或分子结
构。

**Geodesics & Gauges
(测地线和规范)**
表示流形上的结构, 通
常用于处理连续空间的
几何性质, 如在三维形
状分析中





图神经网络的应用领域

分子和材料科学

分子性质预测：通过GNN预测分子的化学性质，如毒性、溶解度、反应活性等。这种方法在药物发现中尤为重要。

药物发现：在化学图中，节点代表原子，边表示化学键。GNN可以在药物发现中生成新分子或优化现有分子结构。

材料设计：GNN可用于预测材料的物理化学性质，帮助设计新材料。

生物信息学

基因相互作用网络：用于预测基因之间的相互作用关系，帮助理解基因功能和疾病机制。

蛋白质结构预测：通过GNN构建蛋白质结构中的原子或氨基酸之间的关系，用于预测蛋白质的三维结构。

疾病预测：通过患者的医疗记录、疾病关系图，GNN可以用于疾病的早期预测和诊断。

物理和工程科学

电网优化：在电网中，节点表示电网中的设备，边表示连接。GNN可以帮助预测电网中的故障、优化电力分配。

流体力学模拟：GNN可以用于流体动力学中的模拟，通过邻接关系来捕捉流体的动态变化。

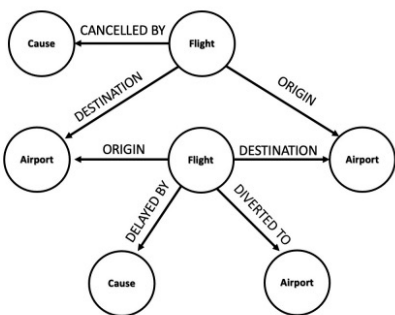
交通网络预测：在交通网络中，GNN用于流量预测和路径优化，帮助进行交通调度和路线规划。

金融风控

反欺诈检测：利用GNN在交易网络中发现潜在的欺诈行为。例如，信用卡交易网络中通过分析用户和商户之间的关系来检测欺诈交易。

信用评分：通过用户之间的关系、交易网络等数据，利用GNN进行信用评估，预测违约风险。

图神经网络的应用领域 (1)

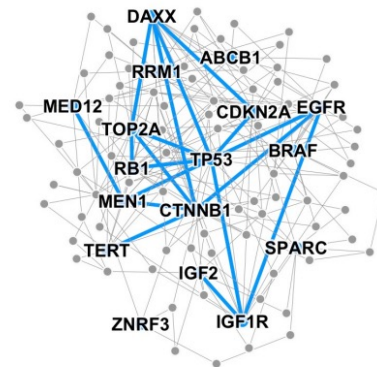


Event Graphs



Image credit: [SalientNetworks](#)

Computer Networks



Disease Pathways

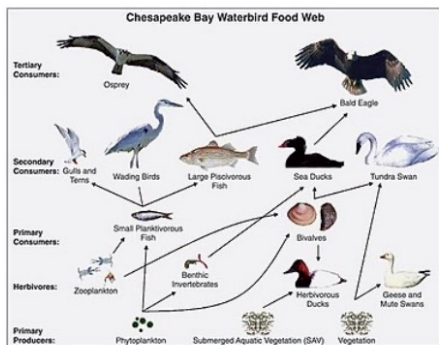


Image credit: [Wikipedia](#)

Food Webs



Image credit: [Pinterest](#)

Particle Networks

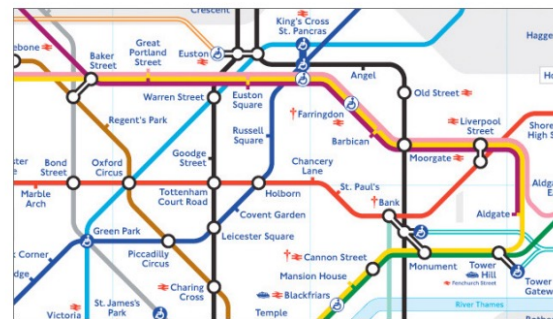


Image credit: [visitlondon.com](#)

Underground Networks



图神经网络的应用领域（2）



Image credit: [Medium](#)

Social Networks

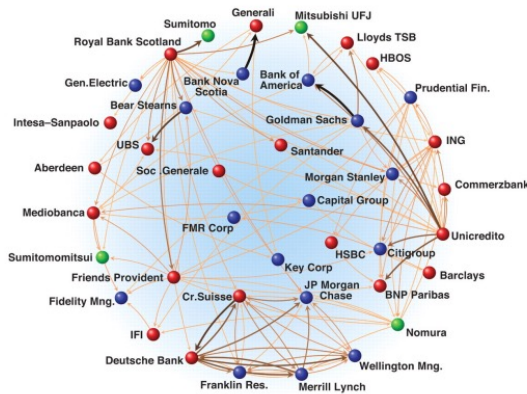


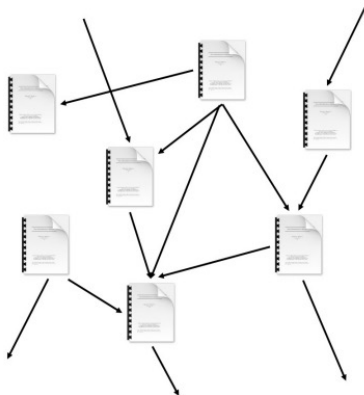
Image credit: [Science](#)

Economic Networks



Image credit: [Lumen Learning](#)

Communication Networks



Citation Networks



Image credit: [Missoula Current News](#)

Internet

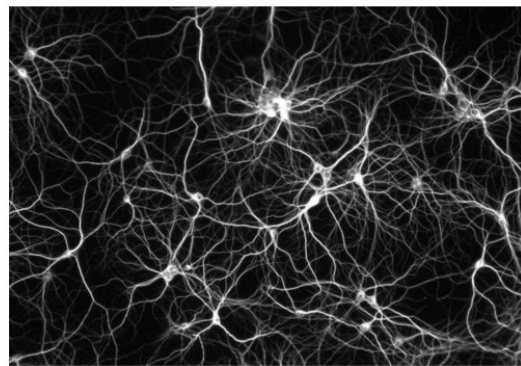


Image credit: [The Conversation](#)

Networks of Neurons

图神经网络的应用领域 (3)

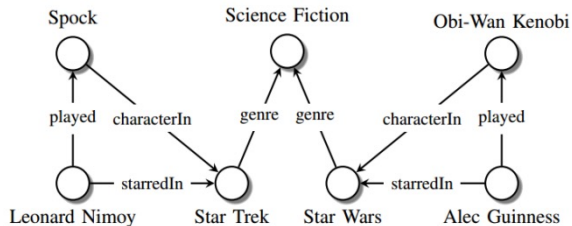


Image credit: [Maximilian Nickel et al](#)

Knowledge Graphs

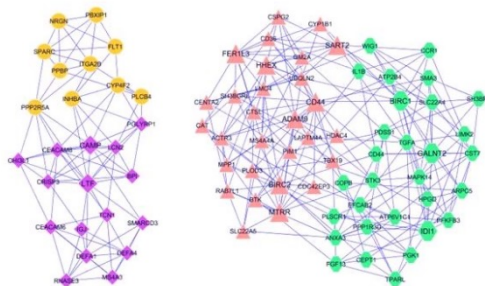


Image credit: [ese.wustl.edu](#)

Regulatory Networks

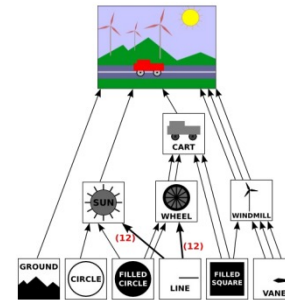


Image credit: [math.hws.edu](#)

Scene Graphs

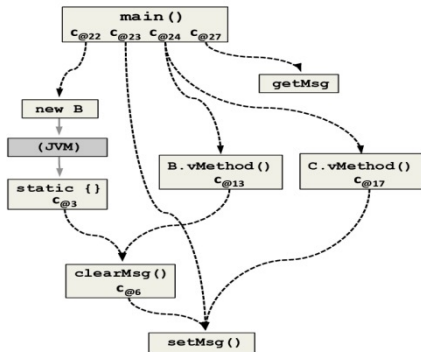


Image credit: [ResearchGate](#)

Code Graphs

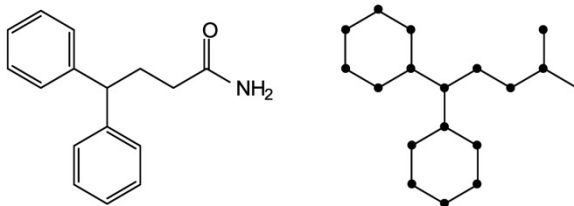


Image credit: [MDPI](#)

Molecules

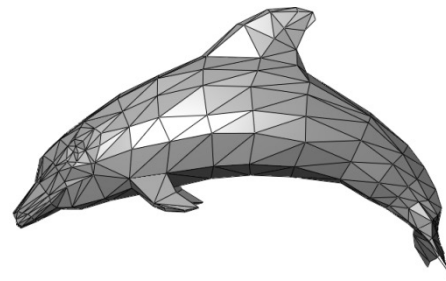


Image credit: [Wikipedia](#)

3D Shapes

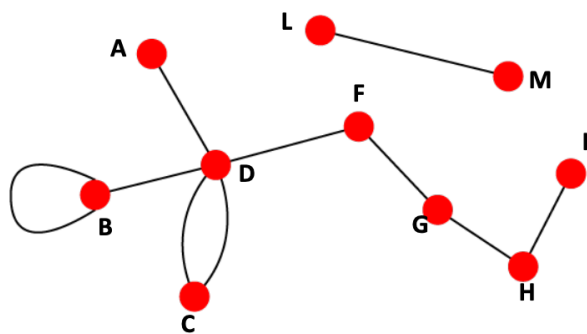
复杂领域具有丰富的关系结构： 可以表示为关系图。

通过显式建模关系： 我们可以获得更好的性能！

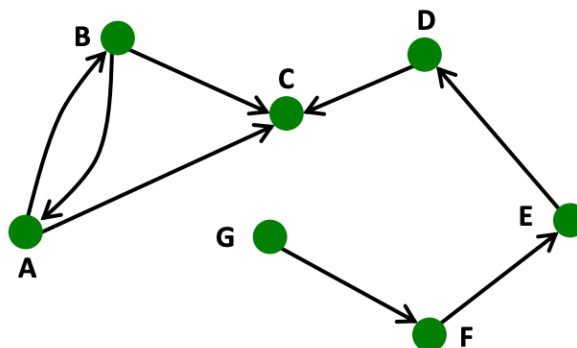
主要问题： 我们如何利用关系结构来实现更好的预测？

图相关的基本概念

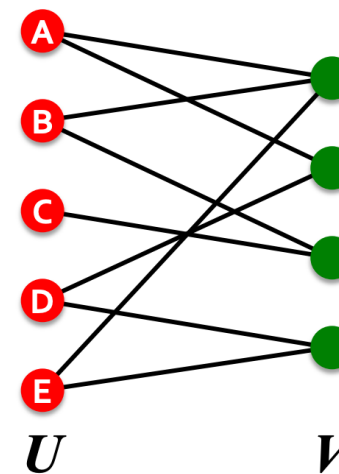
图的类型



无向图



有向图

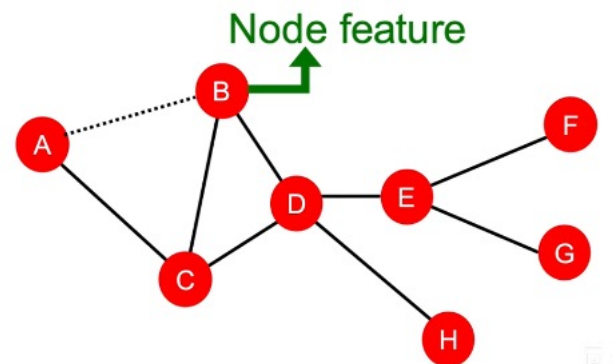


2步图

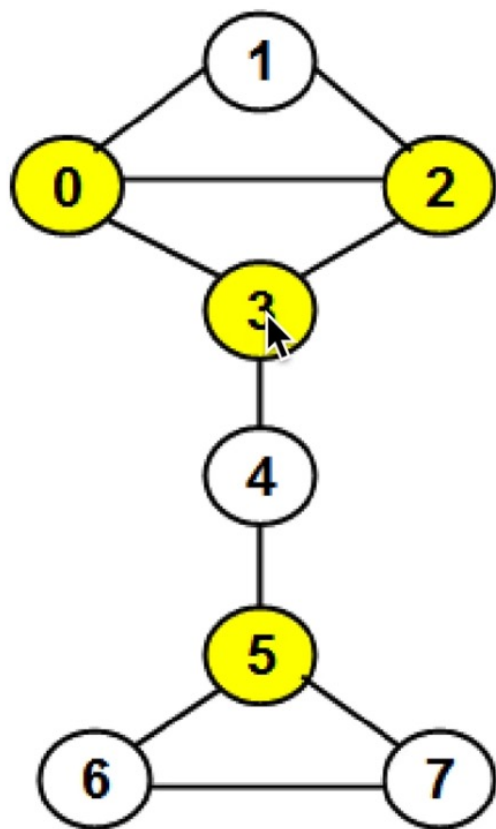
节点特征 @ 图

目标： 一个网络中节点的结构和位置特征表征方法

- **节点度数 (Node degree)**：指的是一个节点所连接的其他节点的数量。
- **节点中心性 (Node centrality)**：用于衡量节点在网络中相对重要的位置，比如衡量该节点在网络中的影响力。
- **聚类系数 (Clustering coefficient)**：表示一个节点的邻居节点彼此连接的概率，即节点所在的局部网络密集程度。
- **图子结构 (Graphlets)**：图中小规模的子结构，用于识别特定节点在网络中的结构角色。



Degree Centrality



$$\begin{array}{c}
 \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\
 \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix} & \times & \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} & = & \begin{bmatrix} 3 \\ 2 \\ 3 \\ 3 \\ 2 \\ 3 \\ 2 \\ 2 \end{bmatrix} \\
 \text{Adjacency Matrix} & & \text{Column Vector} & & \text{Degree Centrality}
 \end{array}
 \end{array}$$

Vertex IDs

0
1
2
3
4
5
6
7



节点中心性 @ Node level feature

1. 节点度数 (Node degree)

计算一个节点的相邻节点数量，但不考虑这些邻居的重要性。“只看数量，不看质量”。

2. 节点中心性 (Node centrality cvc_vcv)

考虑节点在图中的重要性，即不仅关注节点的连接数量，还关注这些连接的质量或重要性。

3. 不同的建模节点重要性的方法：

- 特征向量中心性 (Eigenvector centrality)

- 基于节点的邻居节点的影响力来确定节点的重要性。 求解图的邻接矩阵的特征向量来实现

- 介数中心性 (Betweenness centrality)

- 衡量一个节点出现在其他节点对之间最短路径中的频率。

- 接近中心性 (Closeness centrality)

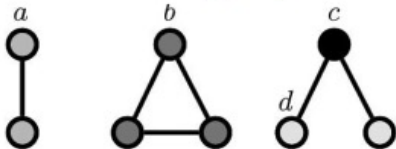
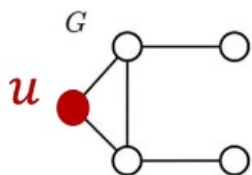
- 衡量节点与其他节点之间的平均最短路径距离，反映节点在图中的传播效率。

Graphlet @ Node level feature

Graphlet Degree Vector (GDV)

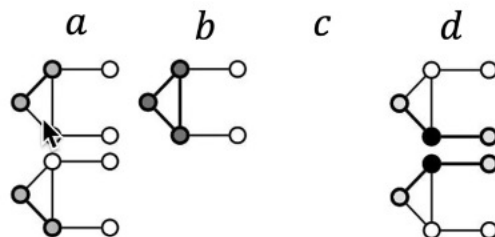
是基于一个给定节点统计其连接的图子结构 (graphlet) 的计数向量

可能的图子结构 (graphlets with 3 nodes)



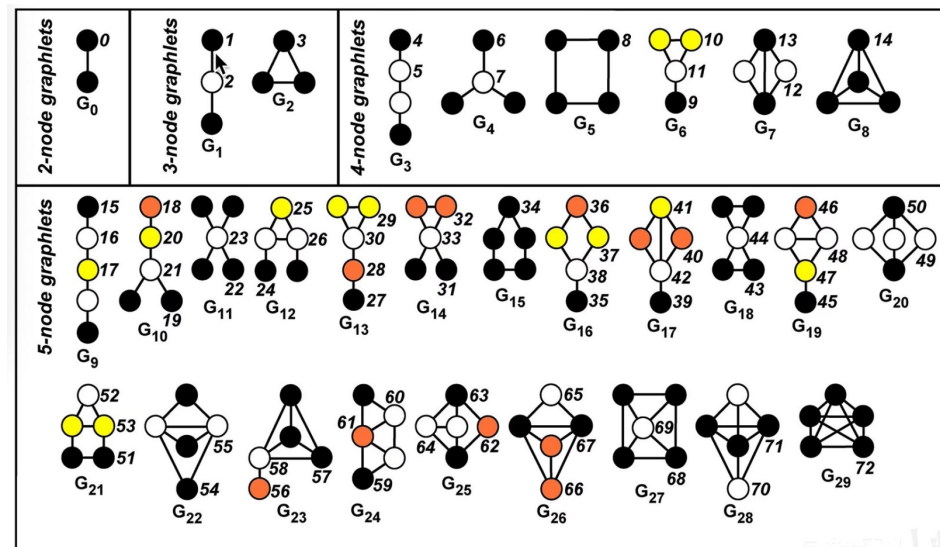
- a: 一个简单的链式结构。
- b: 一个三角形结构。
- c: 一个“叉子”结构。
- d: 一个类似“Y”形的结构。

节点 u 的 GDV 表示



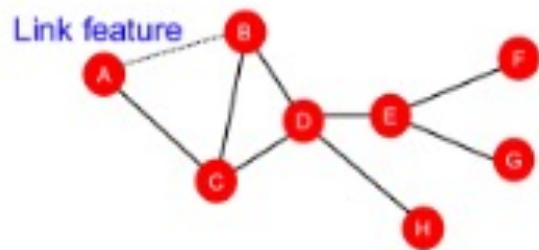
GDV of node u :
 a, b, c, d
 $[2, 1, 0, 2]$

描述了节点 u 的局部邻域拓扑结构信息



同形异构体

连接（边）的特征 @ 图



1. 基于距离的特征 (Distance-based features)

- **定义：**通过计算两个节点之间的最短路径长度来衡量节点之间的关系。
- **局限性：**这种方法只能反映节点间的距离，但无法捕捉节点邻域的重叠情况。
例如，两个节点之间即使距离很短，它们的邻域可能完全不同。

更关注节点间的直接距离

2. 局部邻域重叠 (Local neighborhood overlap)

- **定义：**衡量两个节点共享的邻居节点数量，即两个节点的局部邻域的重叠情况。
- **特点：**当两个节点没有任何共同邻居时，该度量值为零。
- **适用场景：**适合用于分析局部图结构，因为它仅考虑节点的直接邻居，而不涉及全局结构。

只考虑节点的直接邻居，因此在分析局部拓扑关系时更有效

3. 全局邻域重叠 (Global neighborhood overlap)

- **定义：**通过利用整个图的结构来评分两个节点之间的关系。
- **Katz指数：**是一种全局邻域重叠的计算方法，通过计算两节点间的所有路径长度的次数来衡量两节点的相似度。Katz指数考虑了长度不同的路径对相似度的贡献，较短路径的贡献权重重大。
- **适用场景：**适合于需要全局信息的应用场景，因为它综合了图的全局结构，不局限于局部邻域。

考虑整个图的结构信息，使得 Katz 指数等指标可以更全面地反映节点间的关系

连接（边）的特征 @ 图

1. 全局邻域重叠 (Global Neighborhood Overlap)

解决局部邻域度量的限制，通过考虑整个图来衡量节点间的关系

2. Katz Index

- 定义：Katz指数用于计算两个节点之间的相似度，考虑了不同长度的路径的贡献。计算公式如下：

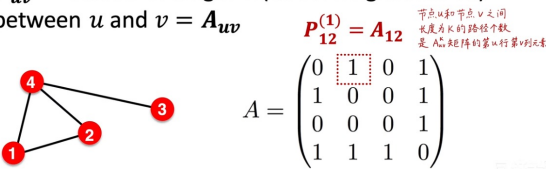
$$S_{v_1, v_2} = \sum_{k=1}^{\infty} \beta^k A_{v_1 v_2}^k$$

其中， β 是衰减因子， $A_{v_1 v_2}^k$ 表示从节点 v_1 到节点 v_2 的长度为 k 的路径数量。

- 路径数量计算：图中展示了计算节点之间路径数量的过程，利用邻接矩阵的幂来计算不同长度的路径。

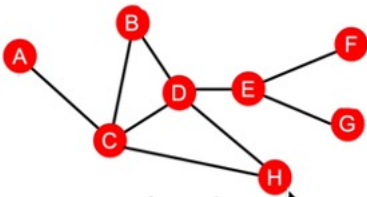
Computing #walks between two nodes

- Recall: $A_{uv} = 1$ if $u \in N(v)$
- Let $P_{uv}^{(K)} = \# \text{walks of length } K \text{ between } u \text{ and } v$
- We will show $P^{(K)} = A^K$
- $P_{uv}^{(1)} = \# \text{walks of length 1 (direct neighborhood) between } u \text{ and } v = A_{uv}$



3. 最短路径距离 (Shortest-path Distance)

- 定义：节点间的最短路径距离是一种简单的距离特征。例如在图中，节点对 (A, B) 的最短距离是 2。
- 局限性：最短路径距离不能很好地捕捉节点邻域的重叠度，例如图中 (B, H) 共享 2 个邻居，而 (E, B) 只共享 1 个邻居。



4. 邻域重叠度量 (Neighborhood Overlap Metrics)

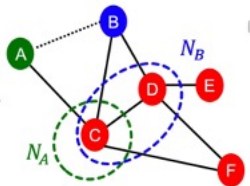
- 共同邻居数 (Common Neighbors)：计算两个节点的共同邻居数量，公式为：

$$|N(v_1) \cap N(v_2)|$$
- Jaccard相似系数 (Jaccard's Coefficient)：定义为共同邻居数占两个节点总邻居数的比例：

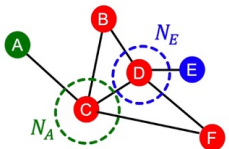
$$J(v_1, v_2) = \frac{|N(v_1) \cap N(v_2)|}{|N(v_1) \cup N(v_2)|}$$

- Adamic-Adar指数：对共同邻居数量进行加权，公式为：

$$\sum_{u \in N(v_1) \cap N(v_2)} \frac{1}{\log |N(u)|}$$



局部邻域特征的限制性



$$N_A \cap N_E = \emptyset$$

$$|N_A \cap N_E| = 0$$

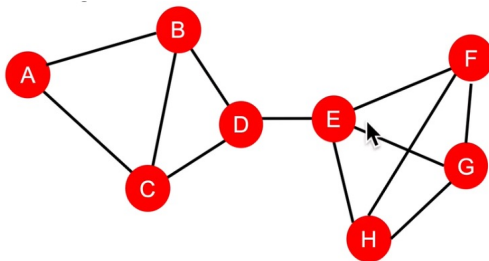
整图级别的特征

目标：我们希望提取能够描述整个图结构的特征。

关键思想：图的词袋模型 (Bag-of-Words for Graphs)

- 回顾 BoW 模型：词袋模型 (BoW) 常用于文本处理，将词语的出现次数作为特征，不考虑词序。
- 在图中应用：可以将图的节点视为“词语”，通过统计不同节点的特征来构建特征向量。
- 问题：简单地将节点当作“词语”有时不足以区分不同的图。例如，如果两个图中都有4个红色节点，它们会得到相同的特征向量，无法有效区分图结构。

$$\phi(\text{Graph 1}) = \phi(\text{Graph 2})$$



节点度的词袋 (Bag of Node Degrees)

Deg1: ● Deg2: ● Deg3: ●

$$\phi(\text{Graph 1}) = \text{count}(\text{Graph 1}) = [1, 2, 1]$$

$$\phi(\text{Graph 2}) = \text{count}(\text{Graph 2}) = [0, 2, 2]$$

Obtains different features for different graphs!

图子结构的词袋 (Bag of Graphlets)

- 定义：给定一个图 G 和图子结构列表 $\mathcal{G}_k = (g_1, g_2, \dots, g_{n_k})$ ，定义图子结构计数向量 f_G ，其中 $(f_G)_i = \#(g_i \subseteq G)$ 表示子结构 g_i 在图 G 中的出现次数。
- 作用：通过统计不同图子结构的出现次数，来构建更加细致的图特征，能够更好地反映图的局部和全局结构信息。

图子结构核方法

- Graphlet Kernel 和 Weisfeiler-Lehman Kernel：这些方法利用了“词袋”表示方式，尤其是针对图的度数或图子结构进行计数，形成特征表示。其中 Weisfeiler-Lehman Kernel 更为复杂，因为它不只是简单地计数节点度，而是通过更复杂的节点属性来描述图结构。



整图级别的特征

Graphlet kernel

vs.

Weisfeller-lehman kernel

限制：图子结构计数的高成本

- **问题：**计算图中的子结构数量非常昂贵。对于一个节点数为 n 的图，大小为 k 的子结构计数需要 $n^{\mathcal{O}(k)}$ 的计算复杂度。
- **原因：**在最坏情况下，这种高复杂度是无法避免的，因为子图同构测试（判断一个图是否为另一个图的子图）是一个 **NP-hard** 问题。
- **算法优化：**如果图的节点度数被限制在 d 以内，那么可以使用一个复杂度为 $\mathcal{O}(nd^{k-1})$ 的算法来计算所有大小为 k 的子结构，但这仍然非常耗时。
- **问题的核心：**如何设计一种更高效的图核来解决这一问题。

- **目标：**通过设计一个高效的图特征描述符 $\phi(G)$ 来简化计算。
- **思路：**使用节点的邻域结构，通过迭代方式逐步丰富节点的特征集。这个方法可以看作是“节点度词袋 (Bag of Node Degrees)”的广义版本，因为节点度仅使用了一跳邻域信息，而这种方法会利用更深层次的邻域信息。

颜色精化 (Color Refinement)

- **算法：**颜色精化 (Color Refinement) 是一种实现这种高效图特征描述的方法。通过颜色精化算法，节点的特征可以被逐步细化，使得最终的特征能够反映更精确的结构信息。

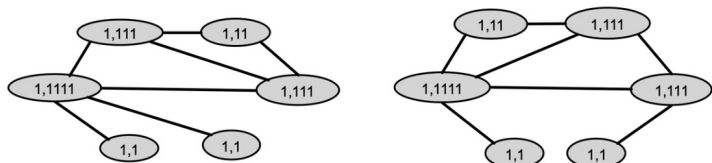
Weisfeller-lehman kernel example

Example of color refinement given two graphs

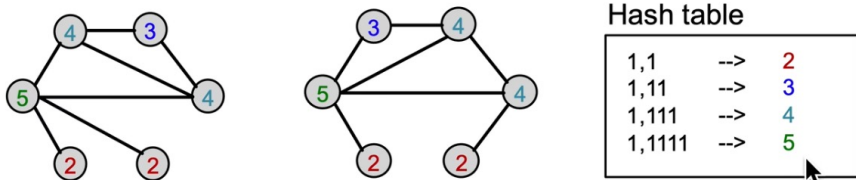
- Assign initial colors



- Aggregate neighboring colors



- Hash aggregated colors

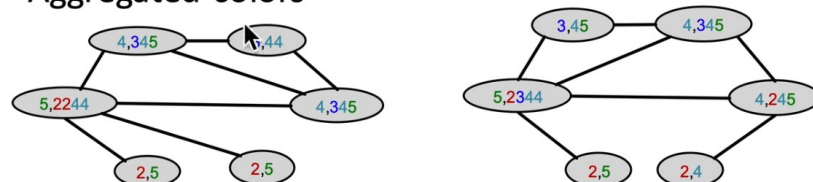


After color refinement, WL kernel counts number of nodes with a given color.

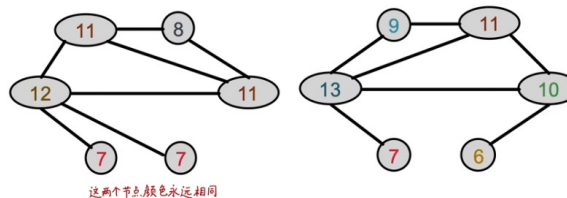
$\phi(\text{Graph 1}) = [6, 2, 1, 2, 1, 0, 2, 1, 0, 0, 0, 2, 1]$
节点个数

$\phi(\text{Graph 2}) = [6, 2, 1, 2, 1, 1, 1, 0, 1, 1, 1, 0, 1]$
某些颜色可能只在一张图中出现

- Aggregated colors



- Hash aggregated colors



Hash table 哈希表由两张图共同贡献

2,4	-->	6
2,5	-->	7
3,44	-->	8
3,45	-->	9
4,245	-->	10
4,345	-->	11
5,2244	-->	12
5,2344	-->	13

The WL kernel value is computed by the inner product of the color count vectors:

$$\begin{aligned}
 K(\text{Graph 1}, \text{Graph 2}) &= \phi(\text{Graph 1})^T \phi(\text{Graph 2}) \\
 &= 49
 \end{aligned}$$



基础概念和定义 @ 几何深度学习

域 (Domain) Ω

- 假设数据存在于一个域 Ω 中
- 例如, 对于图像来说, $u \in \Omega$ 是像素; 对于图结构来说, $u \in \Omega$ 是节点。

特征空间 (Feature Space) $\mathcal{C}$

- 假设存在一个特征空间 $\mathcal{C}$, 用于存储域中元素的特征。在此示例中, $\mathcal{C} = \mathbb{R}^k$, 表示每个元素的特征是一个 k 维向量。

特征化域 (Featurised Domains) :

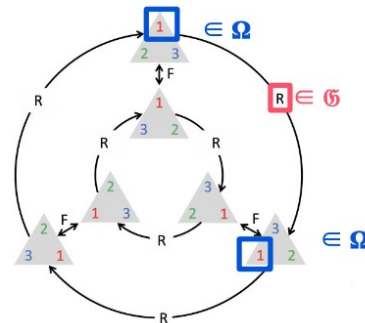
- 可以使用特征函数空间 $\mathcal{X}(\Omega, \mathcal{C})$ 来定义特征化的域。
- 其中, $x \in \mathcal{X}(\Omega, \mathcal{C})$ 是一个函数, 使得 $x(u) \in \mathcal{C}$ 为域 Ω 中元素 u 的特征。
- 对于离散环境, 可以将 $\mathcal{X}$ 视为一个特征矩阵 ($X \in \mathbb{R}^{|\Omega| \times k}$), 表示域 Ω 中的每个元素都有一个 k 维的特征向量。



基础概念和定义 @ 几何深度学习

对称性 (Symmetry) :

- 对称性是一种使对象保持不变 (即**不变性**) 的变换。换句话说, 经过对称变换后, 对象看起来与变换前相同。
- 因此, 对称性必须具备一些性质, 包括:
 - 可组合性 (composable)、
 - 可逆性 (invertible), 以及包含单位元素 (identity)



群的概念:

- 对称性可以通过一种优雅的数学结构来理解, 即**群 (group)**。群为我们提供了一种系统的方法来描述和分析对称性。

群的元素:

- 这些群的元素是域的变换, 例如一些函数 $g: \Omega \rightarrow \Omega$, 它们将域中的元素进行对称变换而不改变对象的性质。

群的定义

- 群是一个集合 G , 配备一个二元运算 $\circ: G \times G \rightarrow G$, 称为组合运算 (composition), 通常简写为 $g \circ h = gh$ 。
- 群必须满足以下四个公理:
 - **结合律 (Associativity)**
对于 $g, h, \ell \in G$, 满足 $(gh)\ell = g(h\ell)$ 。
 - **单位元 (Identity)**
存在一个唯一的元素 $e \in G$, 使得 $eg = ge = g$ 对所有 $g \in G$ 都成立。
 - **逆元 (Inverse)**
对于每个 $g \in G$, 存在唯一的逆元 $g^{-1} \in G$, 使得 $gg^{-1} = g^{-1}g = e$ 。
 - **封闭性 (Closure)**
在组合运算下, 群是封闭的, 即对于每个 $g, h \in G$, 有 $gh \in G$



基础概念和定义 @ 几何深度学习

群作用：

- 关注这些群是如何影响数据的。群作用定义为一个群元素 g 作用在一个域中的元素 u 上，表示为 $(g, u) \rightarrow g \cdot u$
- 举例来说，群作用可以表示图像的平移或旋转，或集合的排列等变换。

线性群作用：

- 特性是保持线性组合的结构，即对于两个向量 x 和 x' ，以及标量 α 和 β ，满足 $g \cdot (\alpha x + \beta x') = \alpha(g \cdot x) + \beta(g \cdot x')$

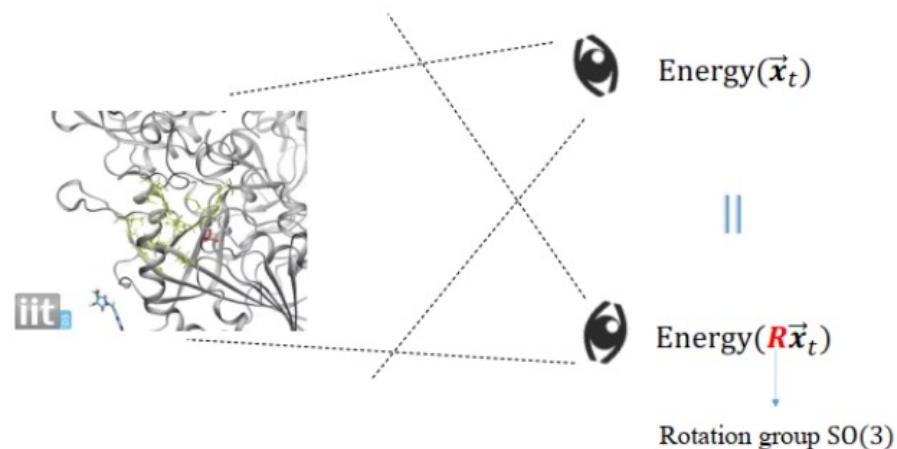
群的表示 (Representation)：

- 群的表示是指将群元素映射到一个可逆矩阵。具体来说，一个 n 维实表示将群 G 中的每个元素 g 映射到一个 $n \times n$ 的可逆矩阵 $\rho(g)$ 。
- 该映射满足 $\rho(g \cdot h) = \rho(g) \cdot \rho(h)$ ，即群的组合性质在矩阵表示中保持不变。
- 如果矩阵 $\rho(g)$ 对所有 $g \in G$ 都是酉矩阵或正交矩阵，则称这个表示为**酉表示 (unitary representation)** 或**正交表示 (orthogonal representation)**。

基础概念和定义 @ 几何深度学习

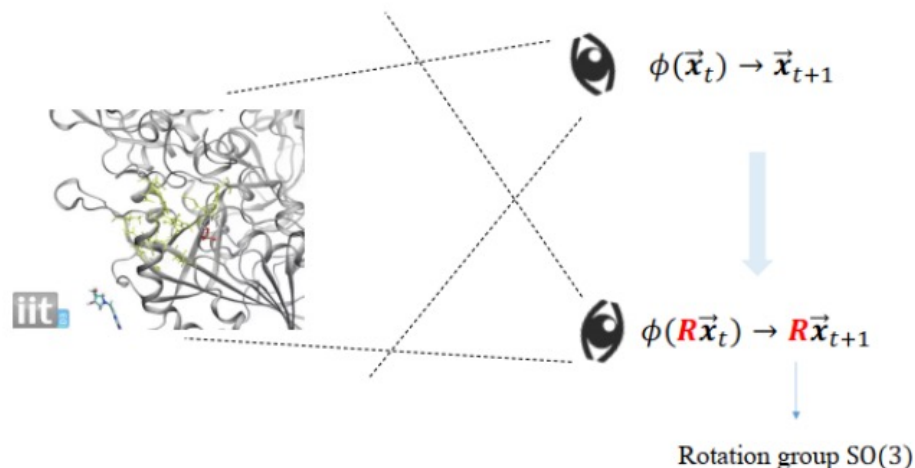
不变形 (Invariance) / 等变性 (Equivariance)

不变性 Invariance



- ☺ 函数 $f: X(\Omega) \rightarrow Y$ 对群 G 是不变的, 若对于所有 $g \in G$, 有 $f(\rho(g)x) = f(x)$ 。这意味着输入在群作用下的变换不会影响函数的输出。示例: 图像分类中的不变性, 分类输出类不应依赖于图像的平移 (如图像内容相同, 只是位置变化)。

等变性 Equivariance



等变性
(Equivariance)
指对一个对象施加一个广义的变换以后, 预测结果是否在某些层面是一致的

$$\begin{aligned} \text{t+1 时刻坐标 } \vec{x}_{t+1} &= \phi(\vec{x}_t) \quad \text{t 时刻坐标} \\ \text{旋转等变性 } R \vec{x}_{t+1} &= \phi(R \vec{x}_t) = R \phi(\vec{x}_t) \end{aligned}$$

- ☺ 函数 $f: X(\Omega) \rightarrow X(\Omega)$ 对群 G 是等变的, 若对于所有 $g \in G$, 有 $f(\rho(g)x) = \rho(g)f(x)$ 。这意味着输入的群作用会以相同方式作用在输出上。示例: 图像分割中的等变性, 分割掩膜应跟随输入图像的平移, 以确保掩膜位置与图像内容一致。

尺度分离 (scale separation)

信号的稳定性:

- 希望信号在域的轻微变形下仍然保持稳定。例如，图像中的对象在轻微的形状变化或位置移动后，其整体内容仍然能够被模型识别。

局部操作的优势:

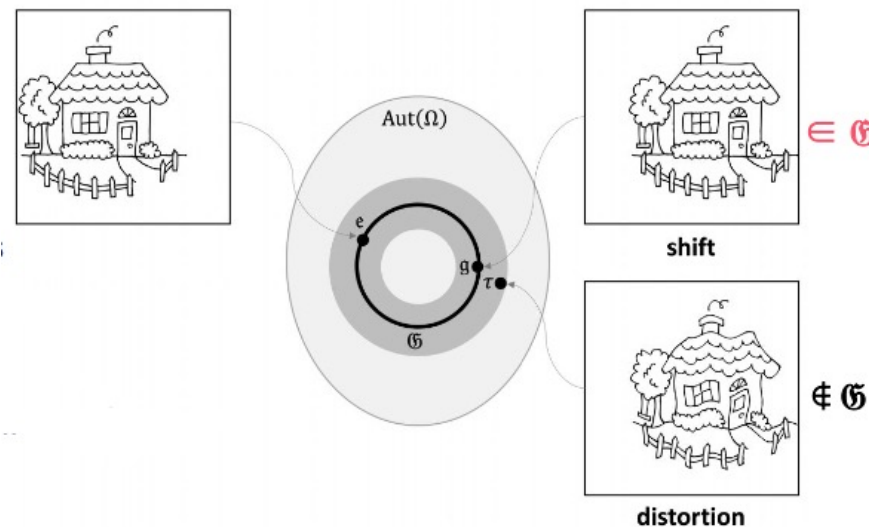
- 使用局部操作组合出大尺度结构非常有益，这样可以避免错误在全局范围内传播。
- 例如，卷积神经网络 (CNN) 通过小的 3×3 卷积核进行深度堆叠，可以有效地捕捉大尺度信息，同时控制局部误差的传播。

支持局部性:

- 因此，设计网络层时应支持局部性，使得每一层都关注局部特征，这样可以更好地处理尺度变化和局部变形。

傅里叶变换 vs 小波变换:

- 傅里叶变换适合处理全局频率信息，而小波变换更擅长在多尺度下进行局部分析，这强调了局部性的重要性。





集合数据 (set)

1. 假设图中没有边：即我们的领域仅包含节点集合 V ，没有边连接。
2. 节点特征表示：令 $\mathbf{x}_i \in \mathbb{R}^k$ 表示第 i 个节点的特征向量，维度为 k 。
3. 构造节点特征矩阵：将所有节点的特征向量堆叠成一个矩阵 $\mathbf{X}$ ，形状为 $n \times k$ ，其中 n 是节点的数量。

$$\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_n)^\top$$

这里，矩阵 $\mathbf{X}$ 的第 i 行对应于节点 i 的特征向量 $\mathbf{x}_i$ 。

4. 节点排序：通过这种方式，我们隐含地对节点进行了排序。然而，我们希望任何神经网络的结果不依赖于这种节点的排序。

$$f \left(\begin{array}{c} \text{X}_5 \\ \text{X}_4 \end{array} \begin{array}{c} \text{X}_1 \\ \text{X}_3 \end{array} \begin{array}{c} \text{X}_2 \end{array} \right) = \mathbf{y} = f \left(\begin{array}{c} \text{X}_2 \\ \text{X}_5 \end{array} \begin{array}{c} \text{X}_1 \\ \text{X}_3 \end{array} \begin{array}{c} \text{X}_4 \end{array} \right)$$



置换/置换矩阵

Permutations and permutation matrices

1. **置换的定义**：可以将改变节点顺序的操作视为置换，置换有 $n!$ 种可能。例如，置换 $(2, 4, 1, 3)$ 表示新的顺序为： $y_1 \leftarrow x_2, y_2 \leftarrow x_4, y_3 \leftarrow x_1, y_4 \leftarrow x_3$ 。
2. **置换矩阵**：在线性代数中，每个置换都可以定义为一个 $n \times n$ 的矩阵，称为置换矩阵。置换矩阵每一行和每一列恰好有一个元素为1，其余元素为0。
3. **置换矩阵的作用**：当置换矩阵左乘特征矩阵 $\mathbf{X}$ 时，可以重新排列 $\mathbf{X}$ 的行，以实现节点顺序的改变。例如，矩阵 $\mathbf{P}_{(2,4,1,3)}$ 左乘 $\mathbf{X}$ 后，将 $\mathbf{X}$ 的行顺序调整为 x_2, x_4, x_1, x_3 。

$$\mathbf{P}_{(2,4,1,3)}\mathbf{X} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \text{---} & \mathbf{x}_1 & \text{---} \\ \text{---} & \mathbf{x}_2 & \text{---} \\ \text{---} & \mathbf{x}_3 & \text{---} \\ \text{---} & \mathbf{x}_4 & \text{---} \end{bmatrix} = \begin{bmatrix} \text{---} & \mathbf{x}_2 & \text{---} \\ \text{---} & \mathbf{x}_4 & \text{---} \\ \text{---} & \mathbf{x}_1 & \text{---} \\ \text{---} & \mathbf{x}_3 & \text{---} \end{bmatrix}$$



置换不变形 (permutation invariance)

1. **置换不变性目标**: 我们希望设计的函数 $f(X)$ 对节点的顺序不敏感。换句话说, 应用置换矩阵不应该改变结果。
2. **定义置换不变性**: 当对于所有的置换矩阵 P , 都有 $f(PX) = f(X)$ 时, 我们称函数 $f(X)$ 是置换不变的。
3. **深度集合模型 (Deep Sets)**: 一种通用的置换不变性模型形式是 Deep Sets 模型, 公式如下:

$$f(X) = \phi \left(\sum_{i \in V} \psi(x_i) \right)$$

其中, ψ 和 ϕ 是可以学习的函数, 例如多层感知机 (MLP)。这里**求和**作为聚合操作是关键, 也可以使用其他选择, 比如最大值 (max) 或平均值 (avg)。

Sum/mean/max pooling/min pooling/sorting/adaptive aggregation



置换等变形 (permutation equivariance)

Permutation Equivariance

1. **置换不变性模型**：置换不变性模型适用于集合级别的输出。
2. **节点级别的输出需求**：如果我们希望获得节点级别的输出，就需要能够识别各个节点的输出。置换不变性聚合器会破坏这种识别能力。
3. **保持节点顺序**：我们希望找到不会改变节点顺序的函数，也就是说，如果我们对节点进行置换，函数前后的顺序不受影响。
4. **置换等变性定义**：当对于所有的置换矩阵 P ，都有 $f(PX) = Pf(X)$ 时，我们称函数 $f(X)$ 是置换等变的。这表示对输入进行的置换在输出上有相同的效果。



图数据 (graph)

1. **节点集扩展为包含边的图**: 现在我们不仅考虑节点, 还要考虑它们之间的边, 即图的边集 $E \subseteq V \times V$ 。
2. **邻接矩阵表示**: 可以使用邻接矩阵 A 来表示这些边, 其中:

$$a_{ij} = \begin{cases} 1, & (i, j) \in E \\ 0, & \text{否则} \end{cases}$$

这里 $a_{ij} = 1$ 表示节点 i 和节点 j 之间存在边。

3. **扩展可能性**: 还可以添加其他信息 (例如边的特征), 但为了简化, 此处忽略这些内容。
4. **置换 (等/不) 变性需求**: 即使在图的上下文中, 主要的需求 (置换不变性或置换等变性) 仍然适用。

置换不变性 / 图结构不变性

$$f \left(\begin{array}{c} \text{X}_5 \quad \text{X}_1 \\ \text{X}_4 \quad \text{X}_2 \\ \text{X}_3 \end{array} \right) = \mathbf{y} = f \left(\begin{array}{c} \text{X}_2 \\ \text{X}_5 \quad \text{X}_4 \\ \text{X}_1 \quad \text{X}_3 \end{array} \right)$$

$$f \left(\begin{array}{c} \text{X}_5 \text{---} \text{X}_1 \\ \text{X}_5 \text{---} \text{X}_2 \\ \text{X}_4 \text{---} \text{X}_2 \\ \text{X}_4 \text{---} \text{X}_3 \end{array} \right) = \mathbf{y} = f \left(\begin{array}{c} \text{X}_2 \text{---} \text{X}_5 \\ \text{X}_2 \text{---} \text{X}_4 \\ \text{X}_1 \text{---} \text{X}_5 \\ \text{X}_3 \text{---} \text{X}_4 \end{array} \right)$$



置换不变性 / 等变性 @ graph

1. **主要区别**：在图结构中，节点的置换也会影响边，因此不仅节点顺序需要调整，边的关系也要对应调整。
2. **邻接矩阵的置换**：在应用置换矩阵 P 时，需要同时对邻接矩阵 A 的行和列进行置换，这可以表示为 PAP^T 。
3. **更新后的函数定义**：
 - **置换不变性 (Invariance)**：如果对于所有置换矩阵 P ，都有 $f(PX, PAP^T) = f(X, A)$ ，则函数 $f(X, A)$ 是置换不变的。
 - **置换等变性 (Equivariance)**：如果对于所有置换矩阵 P ，都有 $f(PX, PAP^T) = Pf(X, A)$ ，则函数 $f(X, A)$ 是置换等变的。



图上的局部性：邻域（neighborhoods）

1. **几何稳定性**：设计几何稳定（局部）等变层对于图学习是非常有利的。
2. **图的局部性**：在图结构中，节点的“邻域”为我们提供了局部性概念。对于节点 i ，其1跳邻域通常定义为：

$$\mathcal{N}_i = \{j : (i, j) \in \mathcal{E} \text{ 或 } (j, i) \in \mathcal{E}\}$$

这里，邻域包括与节点 i 直接相连的所有节点。注意，我们不显式考虑有向边，并且通常假设 $i \in \mathcal{N}_i$ 。

3. **邻域特征集合**：我们可以提取邻域中的特征多重集 $\mathbf{X}_{\mathcal{N}_i} = \{\mathbf{x}_j : j \in \mathcal{N}_i\}$ 。
4. **局部函数定义**：可以定义一个局部函数 g ，作用于该特征多重集 $g(\mathbf{x}_i, \mathbf{X}_{\mathcal{N}_i})$ ，用于处理节点 i 及其邻域的特征。



神经消息传递 (Neural Message Passing)

1. 消息传递 (Message Passing): 每个节点从其邻居节点收集信息。
2. 特征更新 (Feature Update): 节点使用收集到的信息来更新自己的特征表示。

1. 消息传递

对于每个节点 v , 其接收到的消息可以表示为:

$$m_v^{(t)} = \sum_{u \in \mathcal{N}(v)} \text{Message}(h_u^{(t-1)}, h_v^{(t-1)}, e_{uv})$$

其中:

- $\mathcal{N}(v)$ 是节点 v 的邻居节点集合。
- $h_u^{(t-1)}$ 和 $h_v^{(t-1)}$ 是节点 u 和 v 在第 $t-1$ 次迭代的特征表示。
- e_{uv} 是边 (u, v) 的特征 (如果有的话)。
- Message 是一个消息函数, 通常可以是简单的加权和或更复杂的神经网络。

2. 特征更新

节点的特征更新可以表示为:

$$h_v^{(t)} = \text{Update}(h_v^{(t-1)}, m_v^{(t)})$$

其中:

- $m_v^{(t)}$ 是节点 v 在第 t 次迭代接收到的消息。
- Update 是一个更新函数, 通常使用非线性激活函数 (如ReLU) 和可学习的参数。

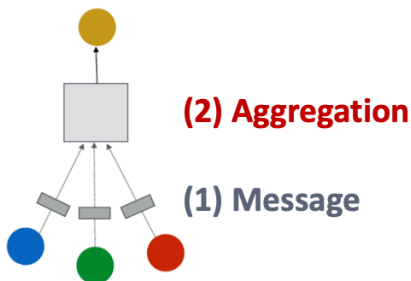
聚合 @ graph

聚合方法是关键步骤之一，它负责将节点的邻居信息整合到目标节点的特征中。不同的聚合方法影响着模型的表现和学习能力。

- **(1) Message:** each node computes a message

$$\mathbf{m}_u^{(l)} = \text{MSG}^{(l)}(\mathbf{h}_u^{(l-1)}), u \in \{N(v) \cup v\}$$
- **(2) Aggregation:** aggregate messages from neighbors

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)}(\{\mathbf{m}_u^{(l)}, u \in N(v)\}, \mathbf{m}_v^{(l)})$$

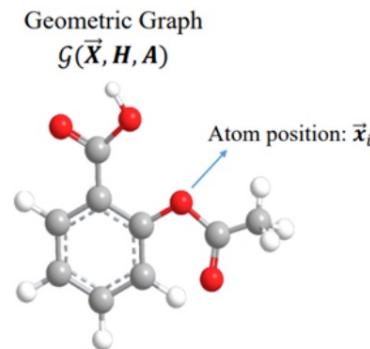
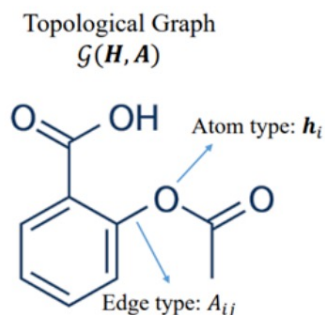


聚合方法	描述	公式	优点
求和聚合	将所有邻居节点的特征相加	$m_v = \sum_{u \in N(v)} h_u$	简单易实现，处理不同邻居数量
平均聚合	对邻居节点特征取平均值	$(m_v = \frac{1}{ N(v) } \sum_{u \in N(v)} h_u)$	
加权聚合	对邻居特征加权求和	$m_v = \sum_{u \in N(v)} w_{uv} h_u$	突出重要邻居的影响
最大聚合	取邻居节点特征的最大值	$m_v = \max_{u \in N(v)} h_u$	捕捉最强信号
注意力聚合	基于注意力机制加权聚合	$m_v = \sum_{u \in N(v)} \alpha_{vu} h_u$	动态调整邻居影响
池化聚合	通过池化层对特征进行聚合	$m = \text{Pooling}(h_u \text{ for } u \in N(v))$	生成图级别特征表示
深度聚合	使用多层神经网络进行聚合	$m_v = \text{MLP}(\sum_{u \in N(v)} h_u)$	增强表达能力
组合聚合	将邻居特征与目标特征拼接处理	$m_v = \text{MLP}(h_v \parallel \sum_{u \in N(v)} h_u)$	提供更丰富信息

拓扑图/几何图/几何拓扑图

拓扑图 $G(H,A)$

- ☺ 在拓扑图中，分子被表示为一个图，其中节点表示原子，边表示原子之间的键。
- ☺ 图的节点属性（也就是原子类型）用 h_i 表示，具体来说就是节点 i 的原子类型，例如氧（O）、氢（H）、碳（C）等。
- ☺ 图的边属性（也就是键类型）用 A_{ij} 表示，即节点 i 和节点 j 之间的键类型，可以是单键、双键等。
- ☺ 拓扑图只包含了原子的类型和键的类型，并不考虑原子在空间中的具体位置。



- Scalars: $H = [h_0, \dots, h_N]$, $A = (A_{ij})_{i=1, j=1}^N$
- Vectors: $\vec{X} = [\vec{x}_0, \dots, \vec{x}_N]$

几何图 $G(X,H,A)$

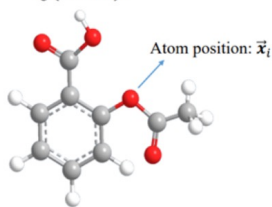
- ☺ 在几何图中，除了拓扑图的信息外，还引入了原子的空间位置信息。
- ☺ 每个节点（原子）的位置用向量 x_i 表示，表示原子在三维空间中的坐标。这使得几何图可以更精确地表达分子结构，因为它包含了每个原子的空间位置信息。

- **Scalars:** $H = [h_0, \dots, h_N]$ ，表示分子中所有原子的类型。
- $A = (A_{ij})_{i=1, j=1}^N$ ，表示分子中所有键的信息，即邻接矩阵。
- **Vectors:** $\vec{X} = [\vec{x}_0, \dots, \vec{x}_N]$ ，表示每个原子的空间位置。

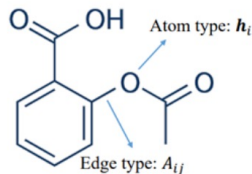


拓扑图/几何图/几何拓扑图

Geometric Graph
 $G(\vec{X}, H, A)$



Topological Graph
 $G(H, A)$



特性	几何图 (Geometric Graph)	拓扑图 (Topological Graph)	几何拓扑图 (Geometric-Topological Graph)
定义	节点和边嵌入在几何空间中，具有明确的坐标和距离	只关注节点间的连接关系，不考虑具体的几何位置	结合几何位置和拓扑结构，同时考虑距离、角度和连接关系
关注点	位置、距离、角度等几何信息	节点和边的连接结构、连通性	几何信息（如距离和角度）与拓扑结构（如连接关系）共同存在
节点位置	有具体的坐标，位置信息很重要	不关心具体位置，只要 连接关系保持	节点有具体坐标，且其几何位置和连接关系同时重要
边的属性	边有具体的长度和方向	边仅表示连接关系，长度和方向无关	边同时具备长度、方向和连接性等属性
边的交叉	边可能会交叉，且交叉点具有明确的几何描述	边的交叉不影响结构分析，交叉无实际几何意义	边的交叉有具体的几何位置，交叉点可用于拓扑分析
变形	不能随意变形，位置、角度和长度信息需要保持	可以自由变形、拉伸，只要保持连接关系	既需要保持几何信息（如长度和角度），也需维持拓扑关系
典型应用	路网、建筑设计、无线通信网络等	社交网络、计算机网络、生物网络、社团关系等	高级道路网分析、复杂分子结构、空间网络中的动态分析等
举例	城市地图、传感器网络	社交网络、计算机网络拓扑	生物分子构象、交通网络动态变化分析、天体轨道网络等
是否可转换	可通过忽略几何信息转化为拓扑图	缺少几何信息，通常无法转化为几何图	不能直接转化，几何信息和拓扑关系共同决定图的结构
数据结构表示形式	$G = (V, E, p)$ ，其中： V 是节点集合， $E \subset V \times V$ ，E是边集合， $p: V \rightarrow R_n$ 为节点的坐标函数，表示节点在 R_n 空间中的位置。通常用邻接矩阵 A 或邻接列表 L 表示连接关系，且包含节点位置 $p(v)$ 信息。	$G = (V, E)$ ，其中： V 是节点集合， $E \subset V \times V$ 是边集合， 不包括任何空间位置信息 。常用邻接矩阵 A，邻接列表 L 或列表 E $E = \{ (v_i, v_j) \mid v_i, v_j \in V \}$ 表示。	$G = (V, E, p, w)$ ，其中： V 是节点集合， $E \subset V \times V$ 是边集合， $p: V \rightarrow R_n$ 为节点的坐标函数， $w: E \rightarrow R$ 表示边的权重或距离。 常用邻接矩阵 A、邻接列表 L，并附带节点坐标 $p(v)$ 和边权重 $w(e)$ 信息。

Building block @ GNN



关键构建模块 @ 几何深度学习

1. 线性 $\mathcal{G}$ -等变层 (Linear $\mathcal{G}$ -equivariant layer) B :

- 这是一个将输入空间 $\mathcal{X}(\Omega, \mathcal{C})$ 映射到输出空间 $\mathcal{X}(\Omega', \mathcal{C}')$ 的线性层, 满足等变性条件 $B(\mathcal{G} \cdot x) = \mathcal{G} \cdot B(x)$, 即群作用在输入上的影响可以通过该层传播到输出上。
- 这在几何深度学习中非常重要, 确保模型对特定的群变换保持一致响应。

Equivariant local layers

2. 非线性 (Nonlinearity) σ :

- 非线性函数 $\sigma : \mathcal{C} \rightarrow \mathcal{C}'$ 逐元素地应用于输入空间, 每个元素 u 的非线性变换表示为 $(\sigma(x))(u) = \sigma(x(u))$ 。
- 非线性是深度学习的关键特性, 使得模型能够学习复杂的映射关系。

Activation function

3. 局部池化 (Local pooling) 或粗化 (Coarsening) P :

- 局部池化操作 $P : \mathcal{X}(\Omega, \mathcal{C}) \rightarrow \mathcal{X}(\Omega', \mathcal{C})$, 其作用是将输入空间的分辨率降低, 例如从 Ω 到 $\Omega' \subseteq \Omega$ 。
- 这种操作有助于提取局部信息, 同时减少数据维度, 从而提升计算效率。

将输入空间降采样
到更小的空间

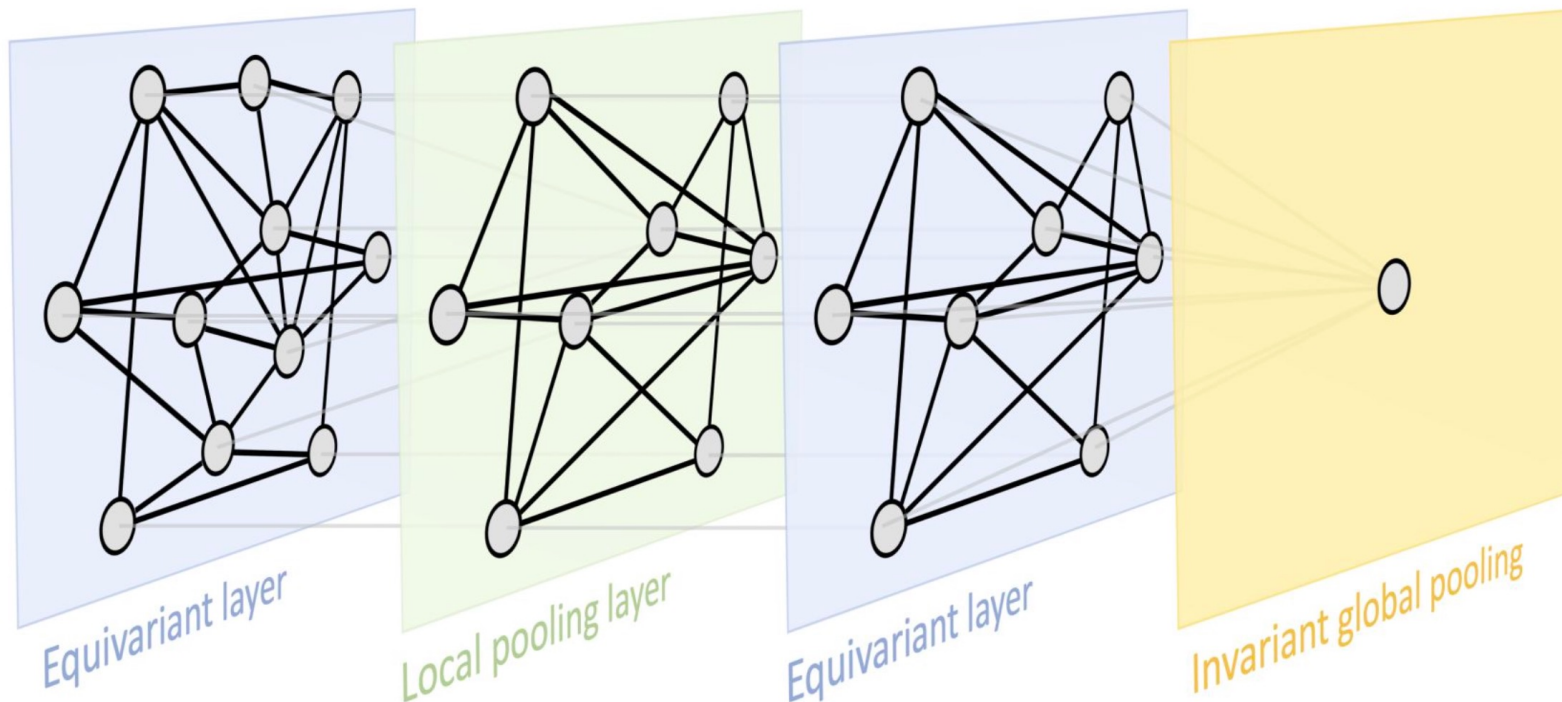
Coarsening layer

4. $\mathcal{G}$ -不变层 ($\mathcal{G}$ -invariant layer) 或全局池化 (Global pooling) A :

- 不变层 $A : \mathcal{X}(\Omega, \mathcal{C}) \rightarrow \mathcal{Y}$ 满足不变性条件 $A(\mathcal{G} \cdot x) = A(x)$, 即群作用在输入上的影响不会影响输出。
- 全局池化用于提取全局特征, 使得模型对群变换保持不变性。

Invariant “tail”

关键构建模块 @ 几何深度学习



等变层

保持输入在特定群作用下的变化模式

使得节点之间的关系和信息在群变换下保持相对一致。每一层的输出仍然是一个图，保留了输入的几何信息，并在群作用下保持一致

局部池化层

通过降低图的分辨率来提取局部信息，减少计算量

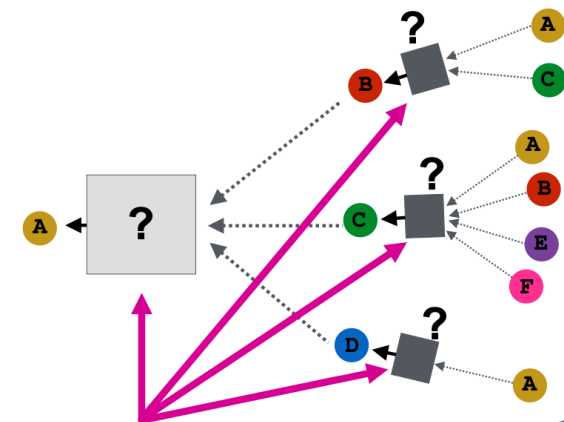
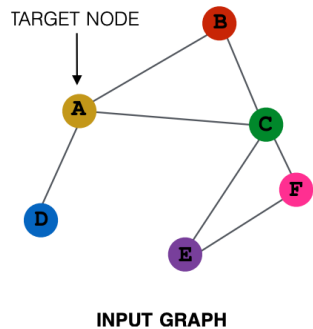
通过减少节点或聚合邻域节点的方式，缩小图的规模，保留了局部特征，从而更有效地捕捉更高层次的特征

等变层

不变全局池化层

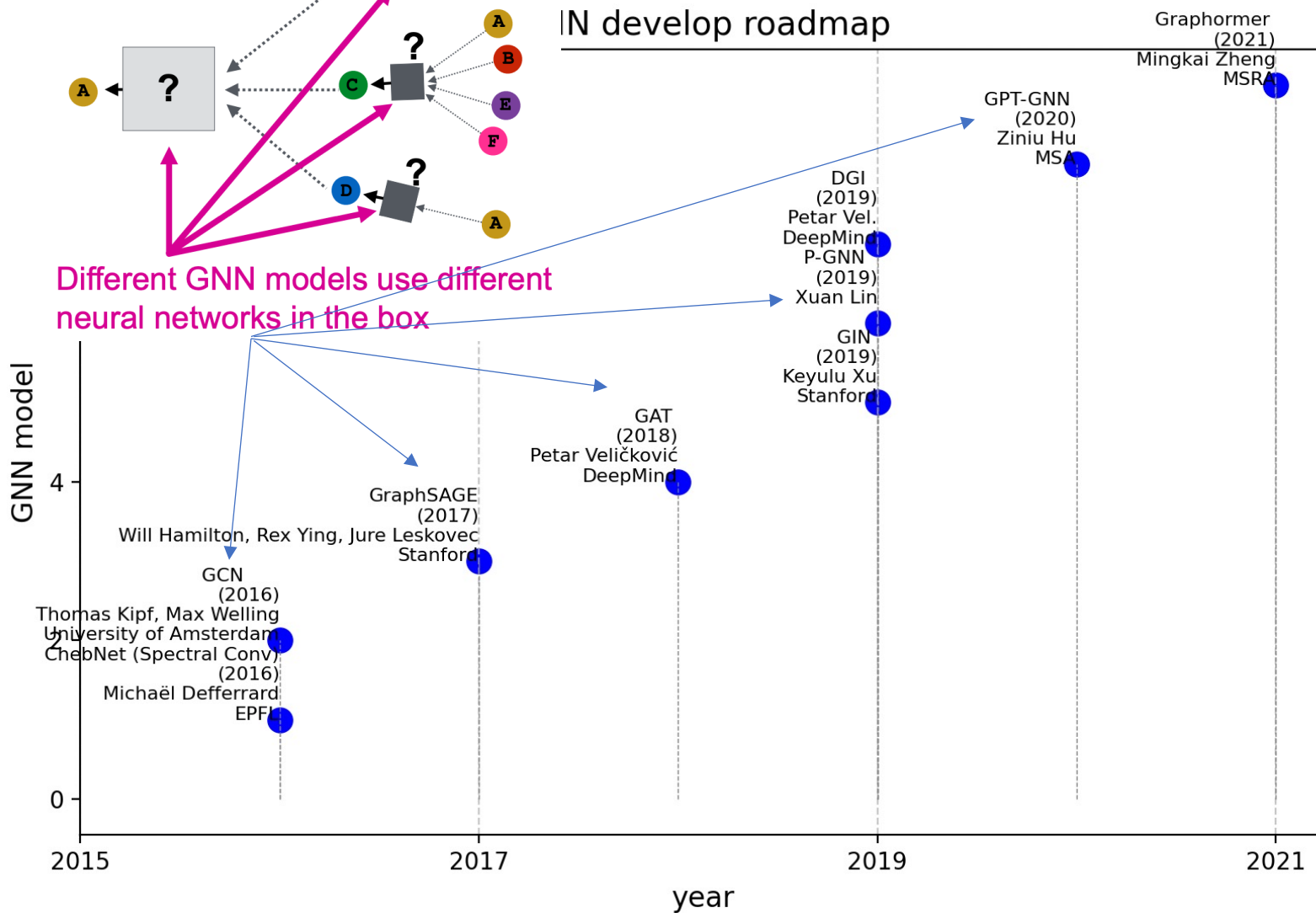
提取全局特征并将图压缩为一个单一的表示

输出对输入的群作用保持不变，使得模型可以获得对称性不变的全局表示



Different GNN models use different neural networks in the box

IN develop roadmap



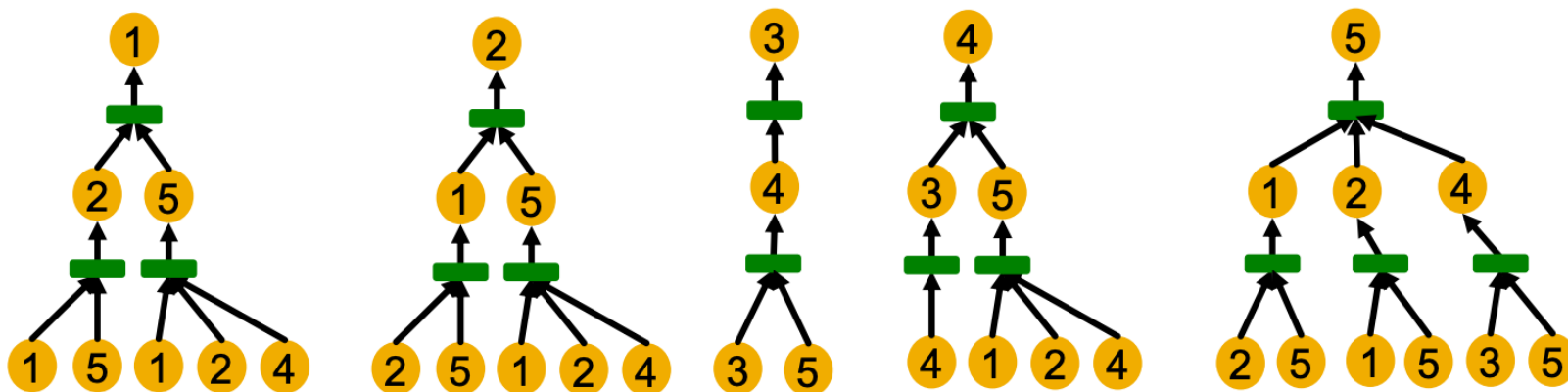


图神经网络的表达力问题

关键构建模块 @ 几何深度学习

表达能力的探讨： 这些GNN模型的表达能力如何？即它们区分不同图结构的能力如何？这反映了GNN在不同任务中识别和处理复杂图结构的有效性。

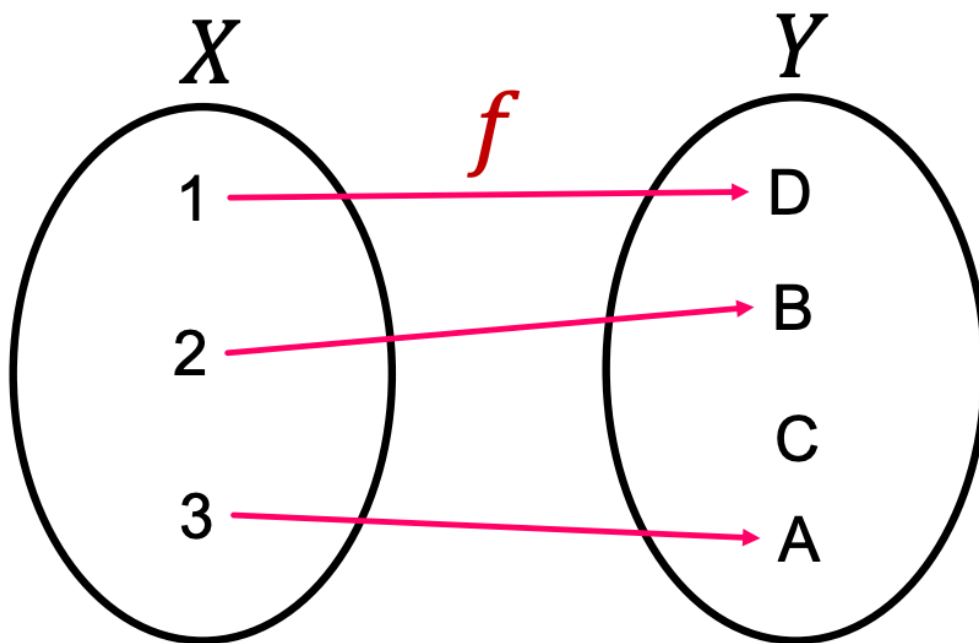
如何设计具备最大表达能力的GNN模型： 思考如何构建一个在表达能力上达到最大化的GNN模型，使其能够更精确地辨别和处理各种图结构。



A classical expressivity test is Graph isomorphism

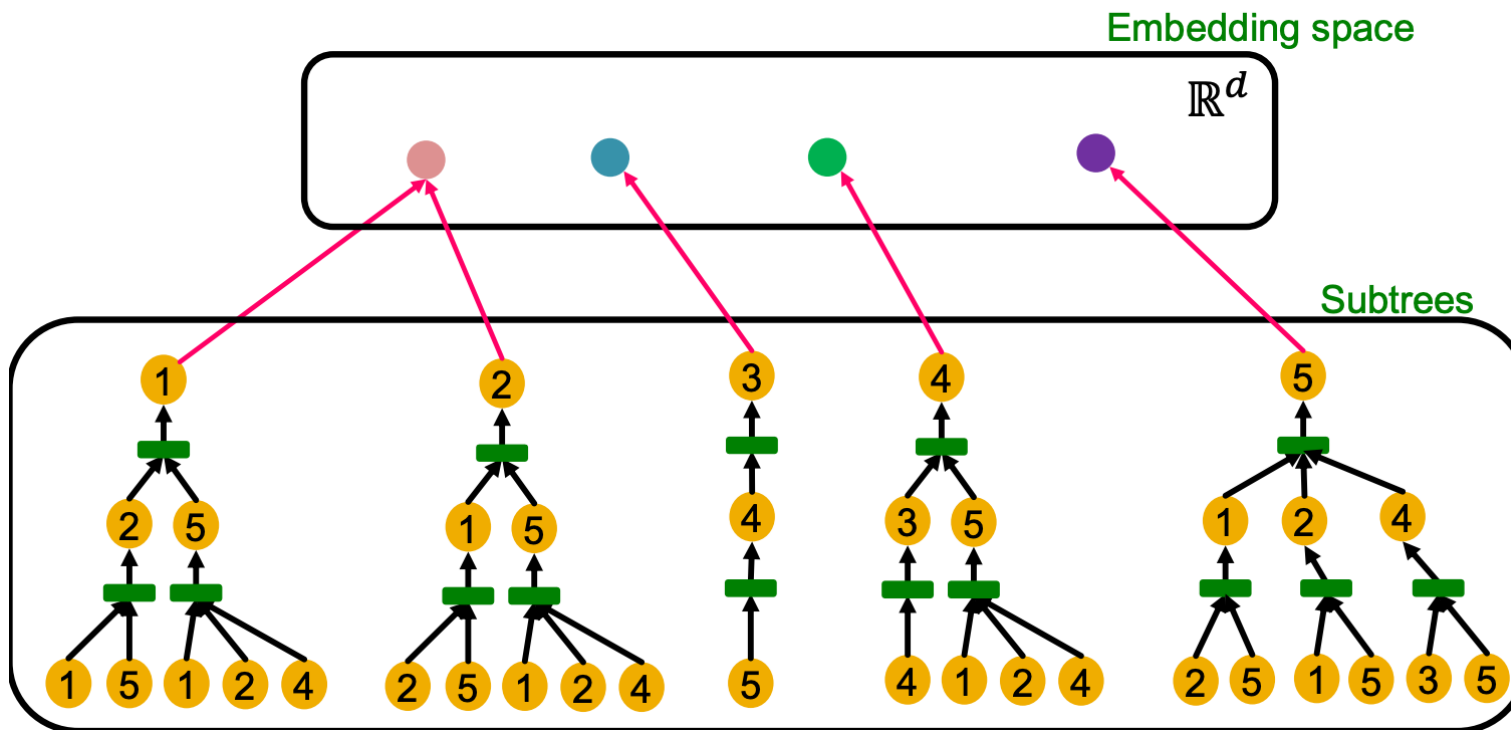
Given a pair of nodes with different neighborhood structures. Is there a GNN that can always tell them apart ?

- **Function** $f: X \rightarrow Y$ is **injective** if it maps different elements into different outputs.
- **Intuition:** f retains all the information about input.



如何来判定一个GNN模型的表达力？

- Most expressive GNN should map subtrees to the node embeddings **injectively**.



Expressive power of GNNs can be characterized by that of neighbor aggregation functions they use.



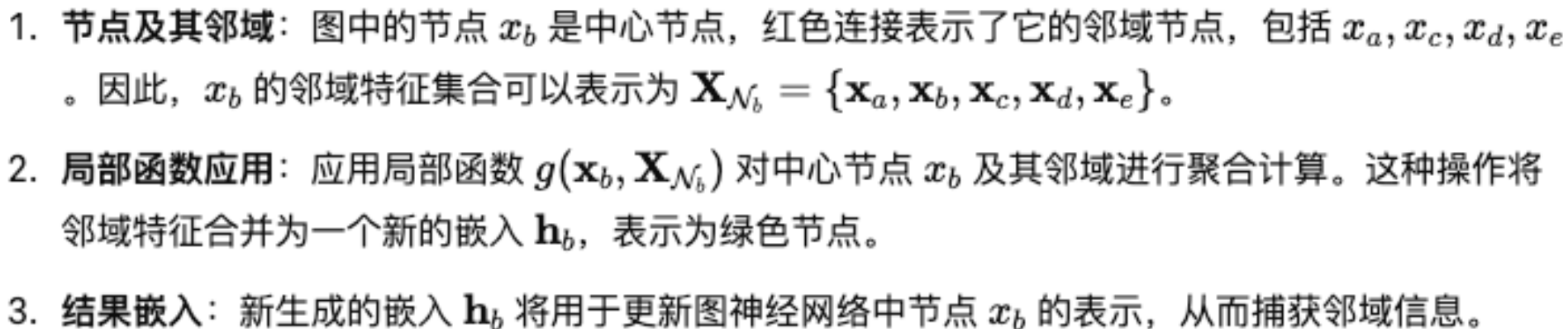
图神经网络



图神经网络的构建方法（GNN）

1. 构建置换等变函数：我们可以通过对每个节点的邻域应用局部函数 g 来构造置换等变函数 $f(X, A)$ 。
2. 函数形式： $f(X, A)$ 的每一行表示为 $g(\mathbf{x}_i, \mathbf{X}_{\mathcal{N}_i})$ ，其中 $\mathbf{x}_i$ 是节点 i 的特征， $\mathbf{X}_{\mathcal{N}_i}$ 是其邻域中的特征集合。对每个节点 i 都应用这个函数 g 。
3. 确保等变性：为了确保等变性，函数 g 不应该依赖于邻域 $\mathbf{X}_{\mathcal{N}_i}$ 中节点的顺序。因此， g 应该是置换不变的。

$$f(\mathbf{X}, \mathbf{A}) = \begin{bmatrix} \text{---} & g(\mathbf{x}_1, \mathbf{X}_{\mathcal{N}_1}) & \text{---} \\ \text{---} & g(\mathbf{x}_2, \mathbf{X}_{\mathcal{N}_2}) & \text{---} \\ & \vdots & \\ \text{---} & g(\mathbf{x}_n, \mathbf{X}_{\mathcal{N}_n}) & \text{---} \end{bmatrix}$$



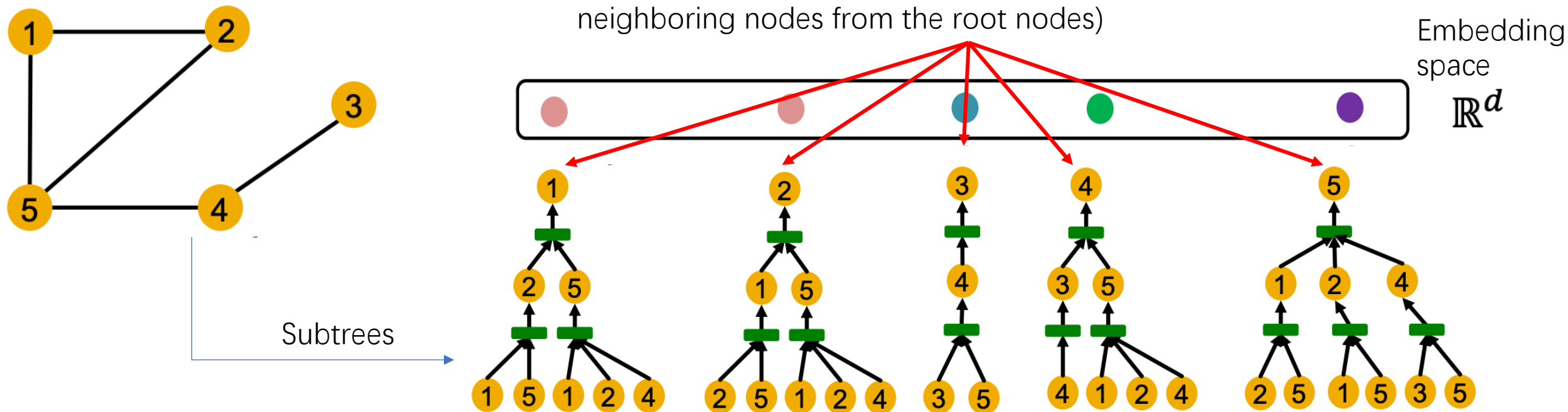
计算图 (computational graph)

different local neighborhoods define
different computational graphs

Computational graphs are identical to rooted
subtree structures around each node

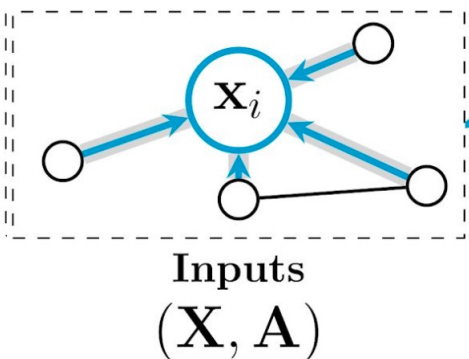
GNN's node embeddings capture
rooted subtree structures

Most expressive GNN maps different rooted
subtrees into different node embeddings

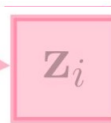
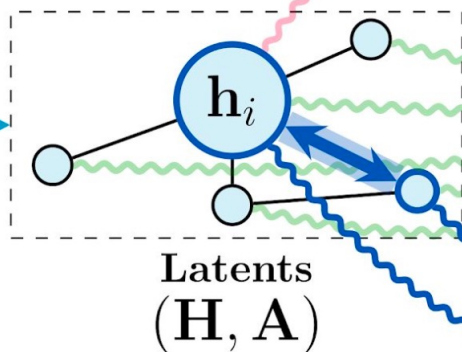


学习任务类型 @ GNN

- **Node-level prediction:** We can directly make prediction using node embeddings!
- After GNN computation, we have **d -dim node embeddings**: $\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\}$
- Suppose we want to make **k -way prediction**
 - Classification: classify among k categories
 - Regression: regress on k targets
- $\hat{\mathbf{y}}_v = \text{Head}_{\text{node}}(\mathbf{h}_v^{(L)}) = \mathbf{W}^{(H)} \mathbf{h}_v^{(L)}$
 - $\mathbf{W}^{(H)} \in \mathbb{R}^{k \times d}$: We map node embeddings from $\mathbf{h}_v^{(L)} \in \mathbb{R}^d$ to $\hat{\mathbf{y}}_v \in \mathbb{R}^k$ so that we can compute the loss



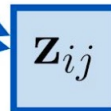
GNN



Node classification
 $\mathbf{z}_i = f(\mathbf{h}_i)$

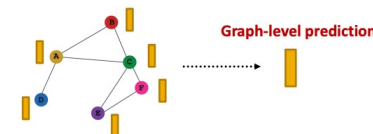


Graph classification
 $\mathbf{z}_G = f(\bigoplus_{i \in V} \mathbf{h}_i)$



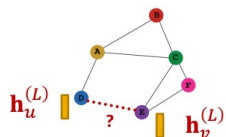
Link prediction
 $\mathbf{z}_{ij} = f(\mathbf{h}_i, \mathbf{h}_j, \mathbf{e}_{ij})$

- **Graph-level prediction:** Make prediction using all the node embeddings in our graph
- Suppose we want to make **k -way prediction**
- $\hat{\mathbf{y}}_G = \text{Head}_{\text{graph}}(\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\})$



- Options for $\text{Head}_{\text{graph}}(\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\})$
- **(1) Global mean pooling**
 $\hat{\mathbf{y}}_G = \text{Mean}(\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\})$
- **(2) Global max pooling**
 $\hat{\mathbf{y}}_G = \text{Max}(\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\})$
- **(3) Global sum pooling**
 $\hat{\mathbf{y}}_G = \text{Sum}(\{\mathbf{h}_v^{(L)} \in \mathbb{R}^d, \forall v \in G\})$

- **Edge-level prediction:** Make prediction using pairs of node embeddings
- Suppose we want to make **k -way prediction**
- $\hat{\mathbf{y}}_{uv} = \text{Head}_{\text{edge}}(\mathbf{h}_u^{(L)}, \mathbf{h}_v^{(L)})$

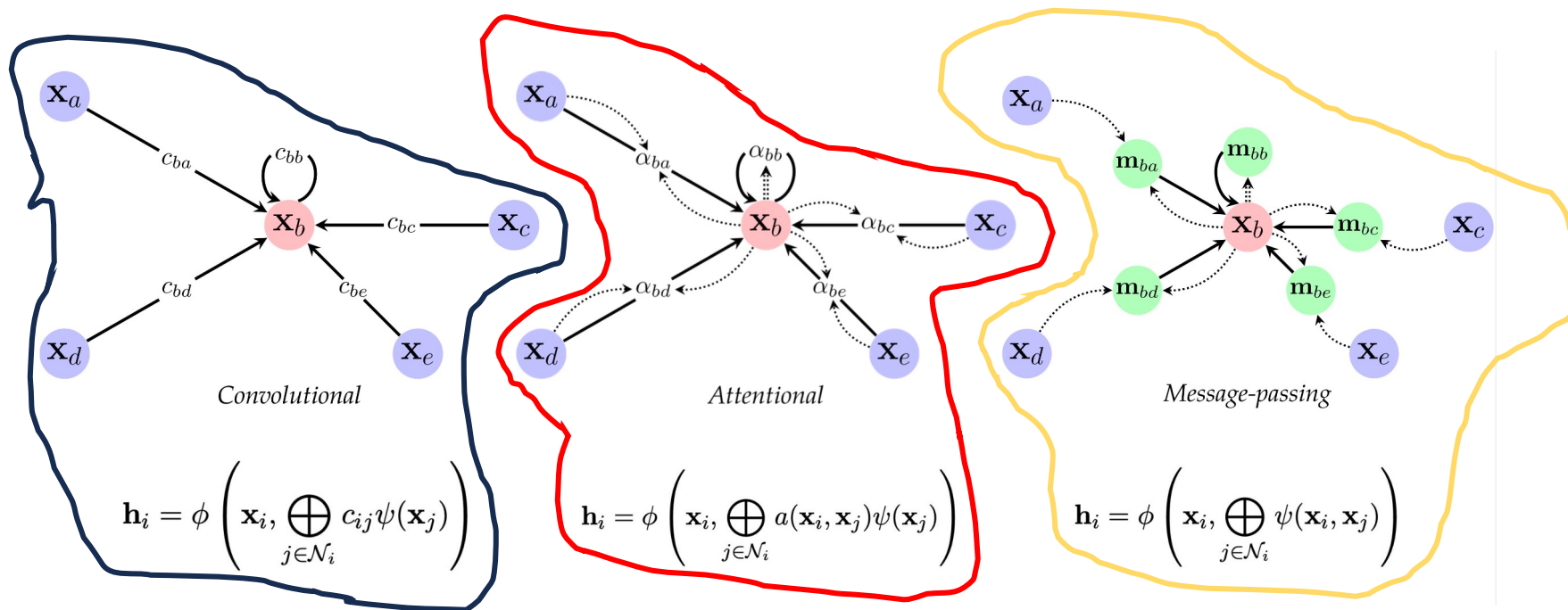


- What are the options for $\text{Head}_{\text{edge}}(\mathbf{h}_u^{(L)}, \mathbf{h}_v^{(L)})$?

节点特征矩阵 $\mathbf{X}$
邻接矩阵 $\mathbf{A}$

消息聚合形式 @ GNN

A more expressive aggregation function leads to a more expressive GNN.
Injective aggregation function leads to the most expressive GNN.



卷积型 (Convolutional):

- 方法: 使用一个权重 c_{ij} 聚合邻居节点的特征。
- 公式: 聚合表示为 $\mathbf{h}_i = \phi \left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} c_{ij} \psi(\mathbf{x}_j) \right)$ 。
- 示例: GraphSAGE、GCN、SGC。
- 解释: 每个节点通过固定的权重 (例如邻接矩阵的权重) 从其邻居接收信息。

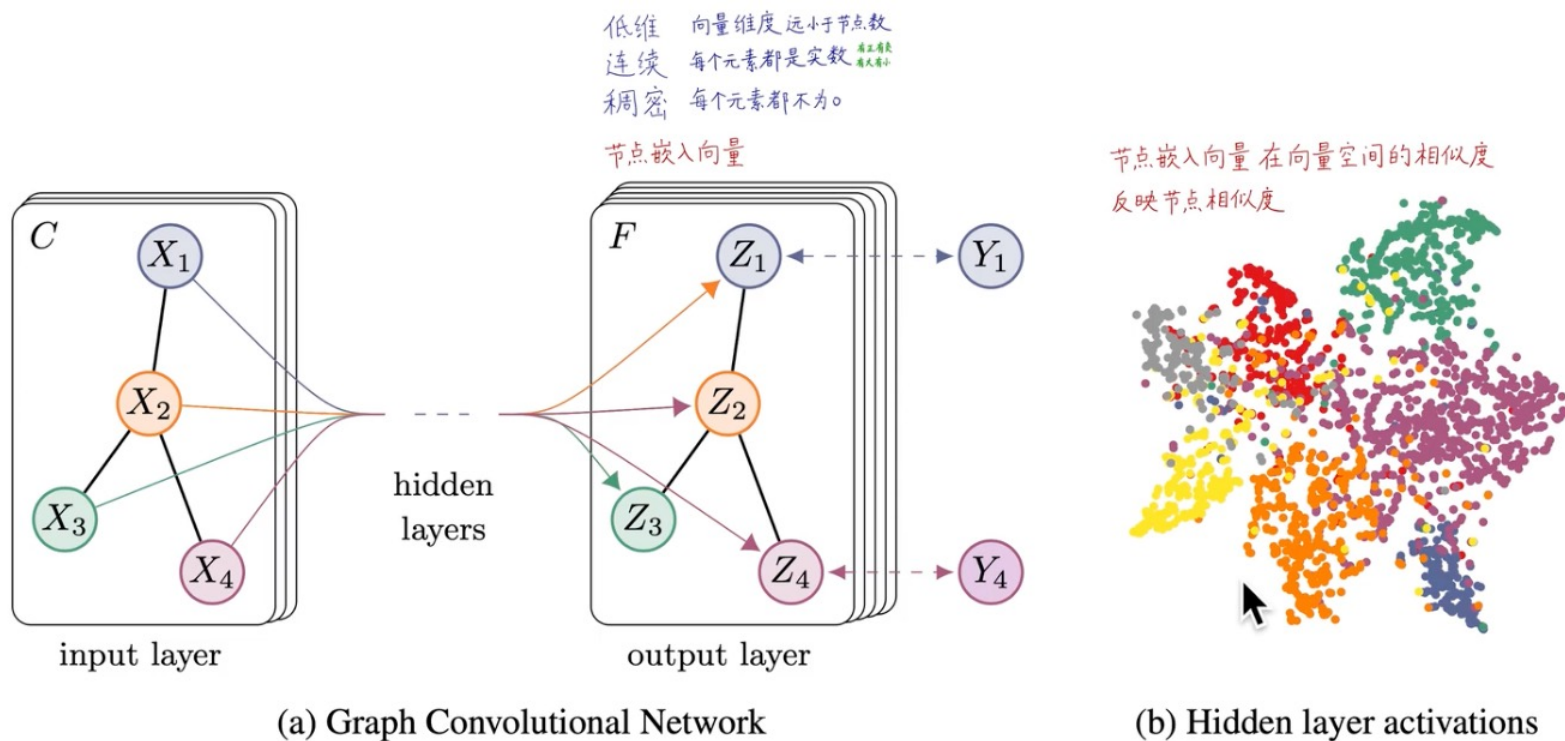
注意力型 (Attentional):

- 方法: 使用注意力机制, 计算每个邻居对中心节点的影响力 (注意力权重 α_{ij})。
- 公式: $\mathbf{h}_i = \phi \left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} a(\mathbf{x}_i, \mathbf{x}_j) \psi(\mathbf{x}_j) \right)$, 其中 $a(\mathbf{x}_i, \mathbf{x}_j)$ 是注意力权重。
- 示例: MoNet、GAT、GATv2。
- 解释: 通过自适应的注意力权重, 节点从不同邻居接收不同程度的影响。

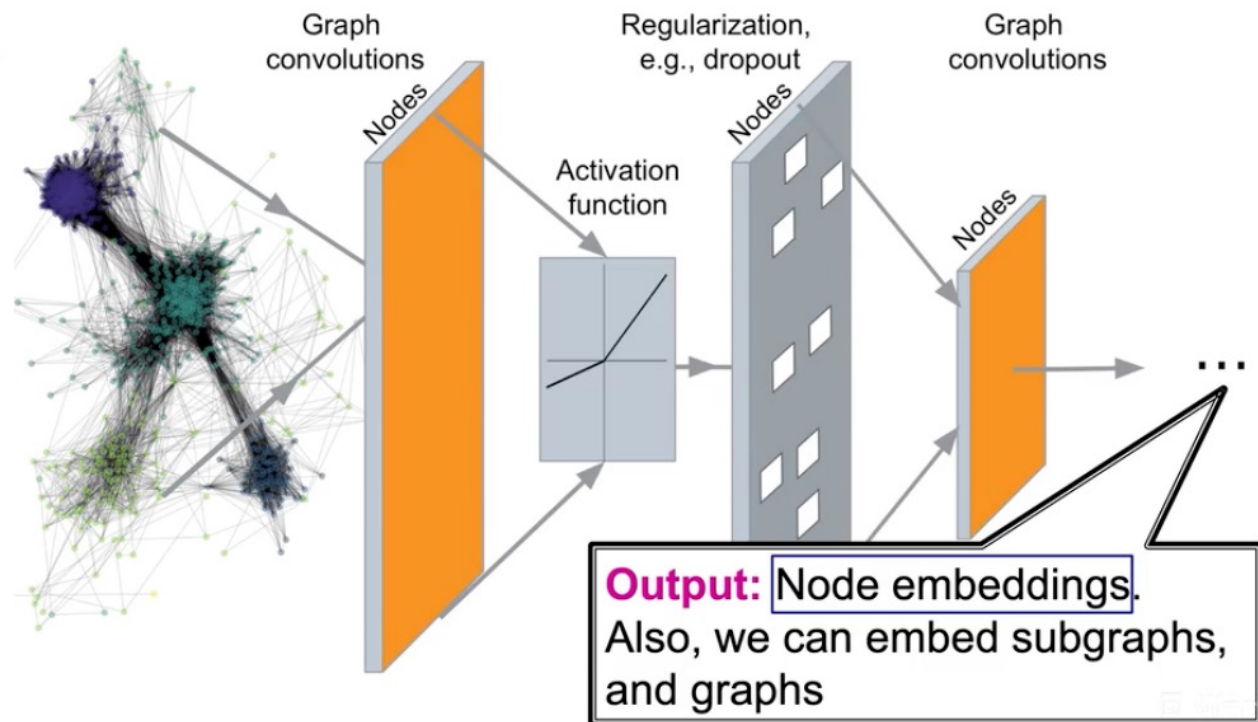
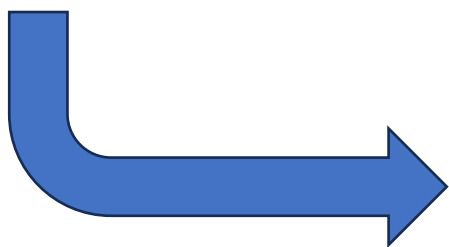
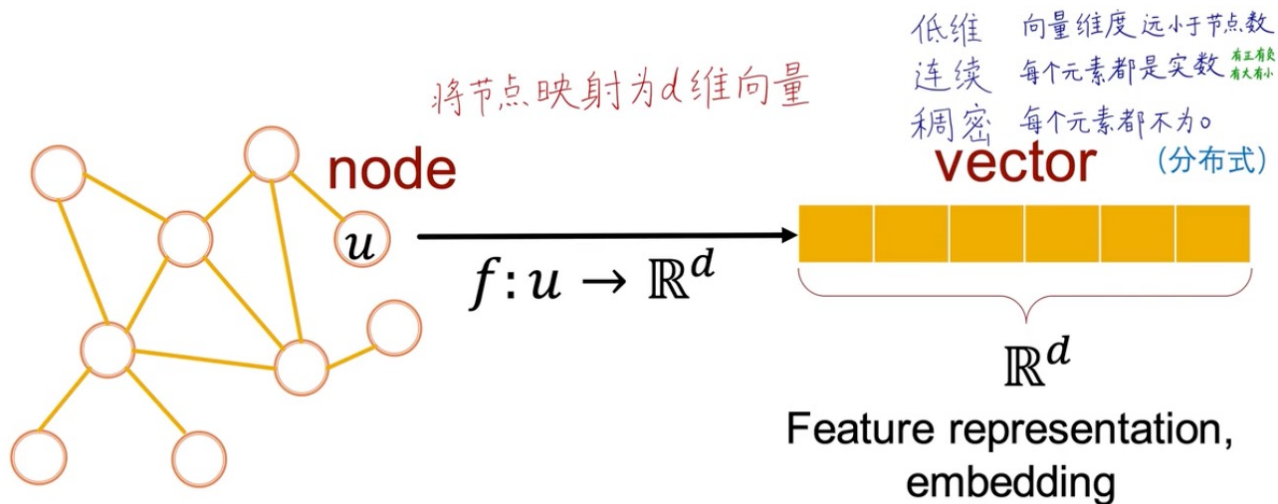
消息传递型 (Message-passing):

- 方法: 每个邻居节点发送消息 m_{ij} 给中心节点, 消息包含邻居节点的信息。
- 公式: $\mathbf{h}_i = \phi \left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} \psi(\mathbf{x}_i, \mathbf{x}_j) \right)$ 。
- 示例: IN、MPNN、GraphNet。
- 解释: 通过消息传递的方式, 节点根据邻居的消息来更新自身状态, 消息可以依赖于边特征。

图卷积神经网络构建方法

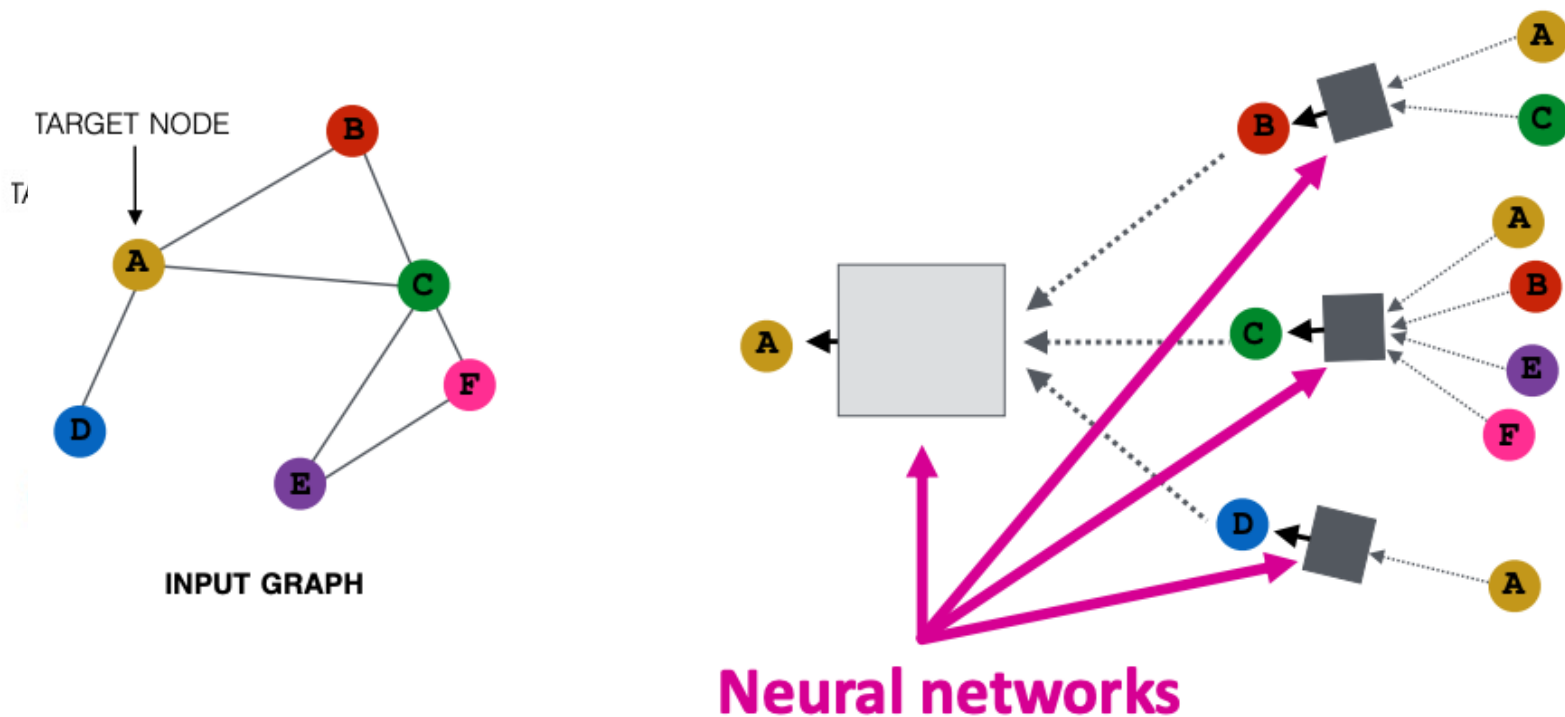


图卷积神经网络



Idea : 邻域聚合

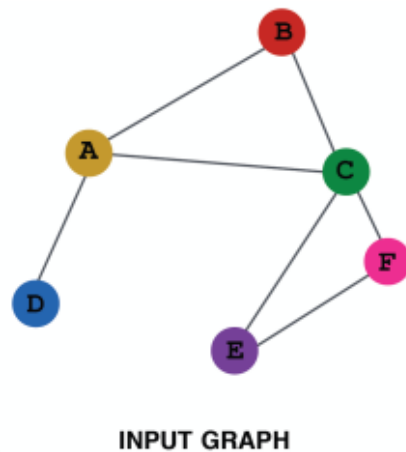
- **Intuition:** Nodes aggregate information from their neighbors using neural networks



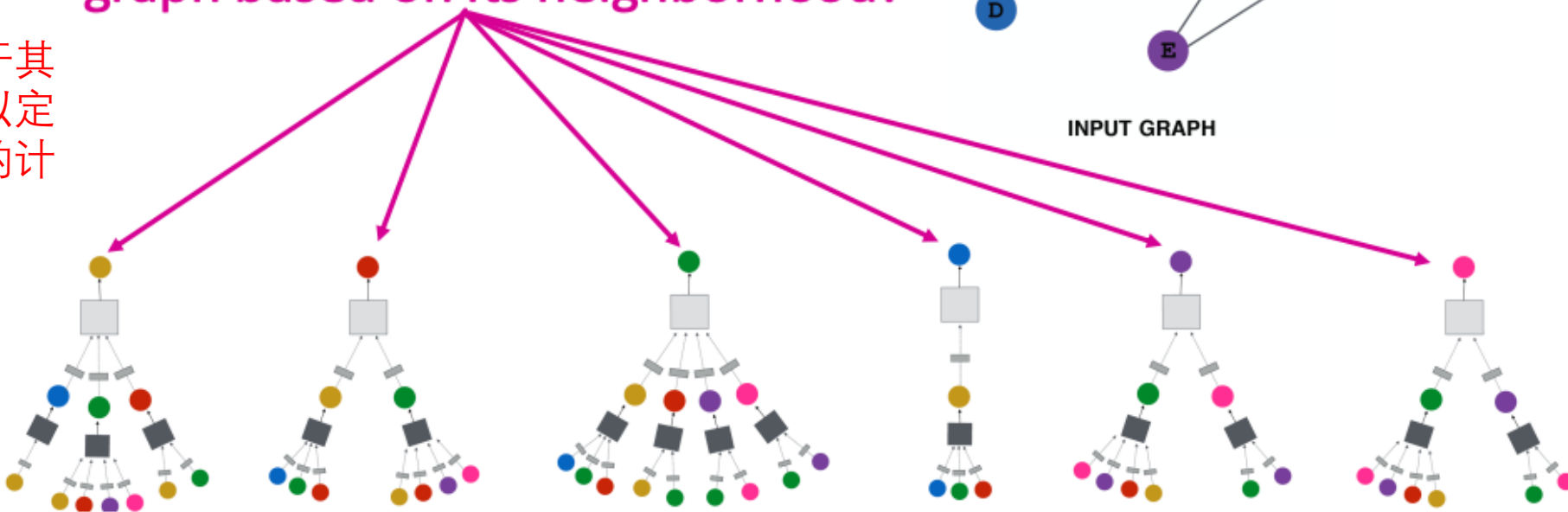
Idea : 邻域聚合

- **Intuition:** Network neighborhood defines a computation graph

Every node defines a computation graph based on its neighborhood!



每个节点基于其邻居节点可以定义一个独立的计算图

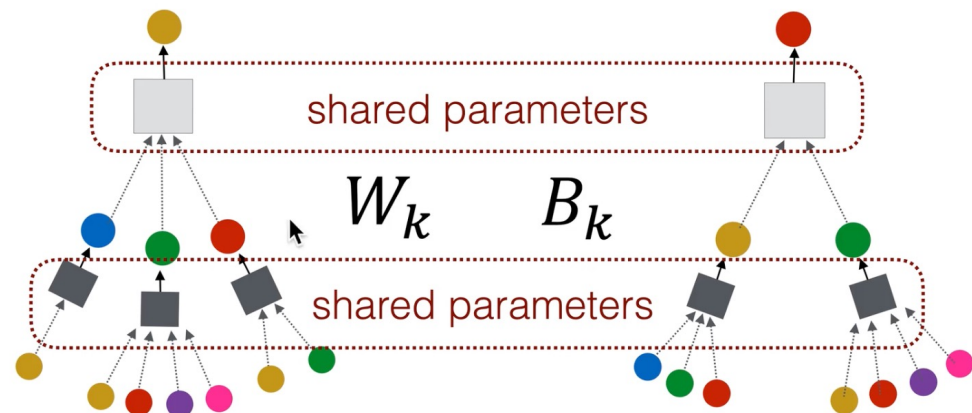
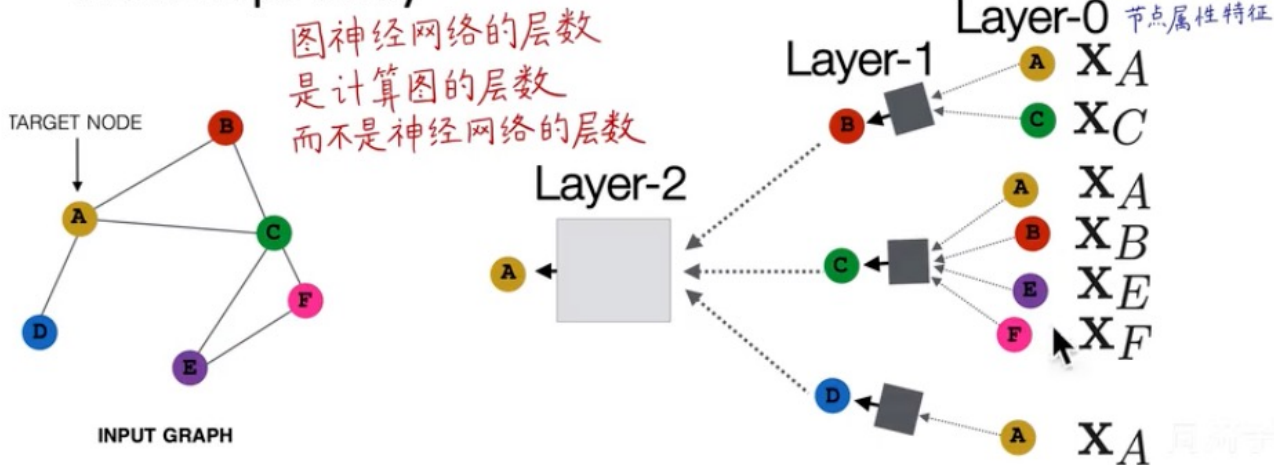


- Model can be **of arbitrary depth**:
 - Nodes have embeddings at each layer
 - Layer-0 embedding of node v is its input feature, x_v
 - Layer- k embedding gets information from nodes that are k hops away

理论上任意深度

“六度空间”理论

图神经网络的层数
是计算图的层数
而不是神经网络的层数

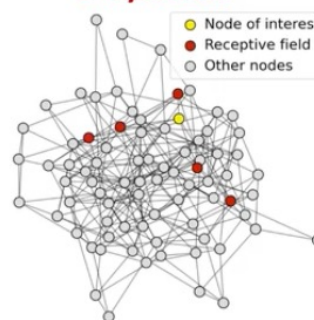


Compute graph for node A

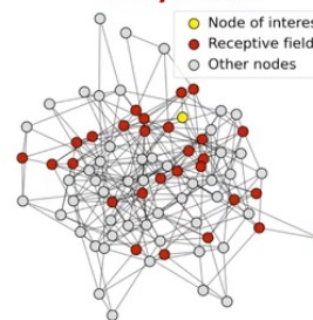
Compute graph for node B

- Receptive field**: the set of nodes that determine the embedding of a node of interest
 - In a K -layer GNN, each node has a receptive field of K -hop neighborhood

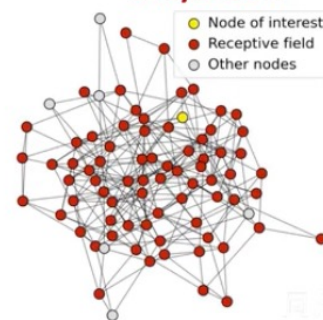
Receptive field for 1-layer GNN



Receptive field for 2-layer GNN



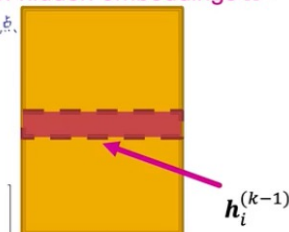
Receptive field for 3-layer GNN



- **Basic approach:** Average information from neighbors and apply a neural network

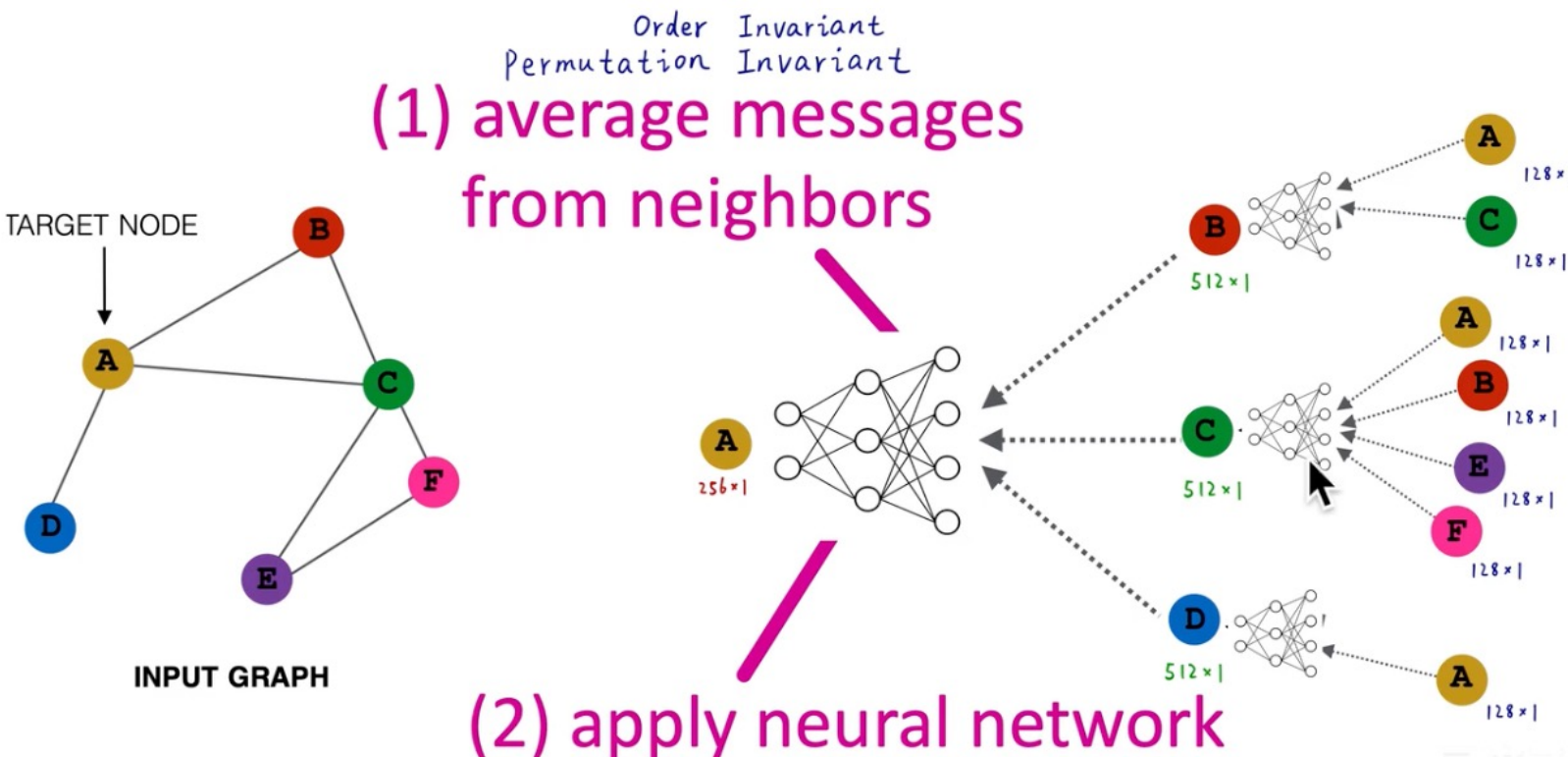
- Many aggregations can be performed efficiently by (sparse) matrix operations

- Let $H^{(k)} = [h_1^{(k)} \dots h_V^{(k)}]^T$
- Then: $\sum_{u \in N_v} h_u^{(k)} = A_{v,:} H^{(k)}$ 选出v节点邻域节点. Embedding
- Let D be diagonal matrix where $D_{v,v} = \text{Deg}(v) = |N(v)|$
 - The inverse of D : D^{-1} is also diagonal:
 $D_{v,v}^{-1} = 1/|N(v)|$
 $D = \begin{bmatrix} d(1) & 0 & 0 & \dots & 0 \\ 0 & d(2) & 0 & \dots & 0 \\ 0 & 0 & d(3) & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & d(n) \end{bmatrix}$
 $D^{-1} = \begin{bmatrix} \frac{1}{d(1)} & 0 & 0 & \dots & 0 \\ 0 & \frac{1}{d(2)} & 0 & \dots & 0 \\ 0 & 0 & \frac{1}{d(3)} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \frac{1}{d(n)} \end{bmatrix}$
- Therefore,



$$\sum_{u \in N(v)} \frac{h_u^{(k-1)}}{|N(v)|} \longrightarrow D^{-1} A H^{(k)}$$

Row Normalized Matrix



$$h_v^{(0)} = x_v$$

$$h_v^{(k+1)} = \sigma \left(W_k \left(\sum_{u \in N(v)} \frac{h_u^k}{|N(v)|} \right) \right)$$

$$z_v = h_v^K$$

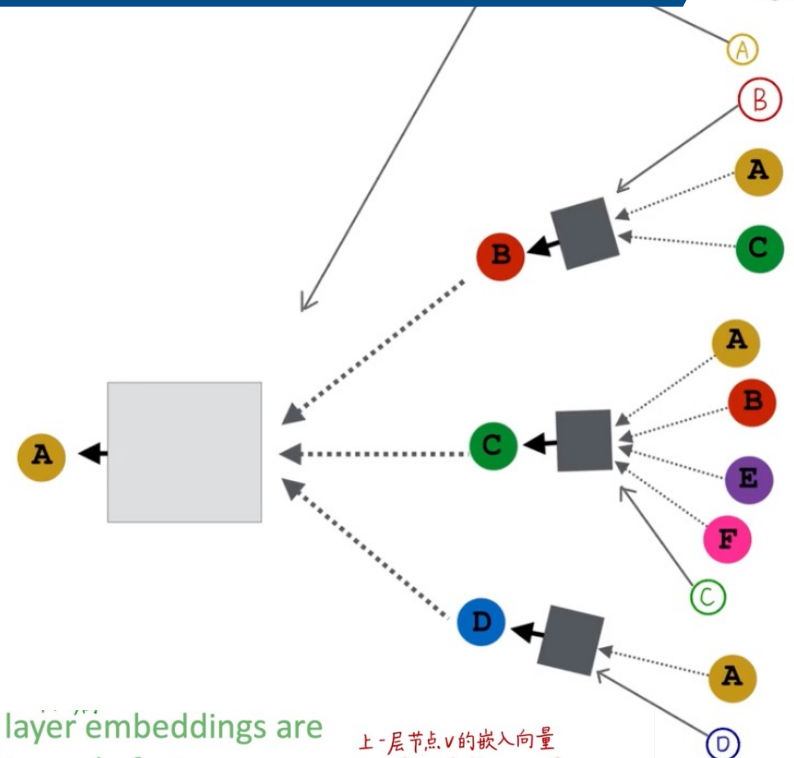
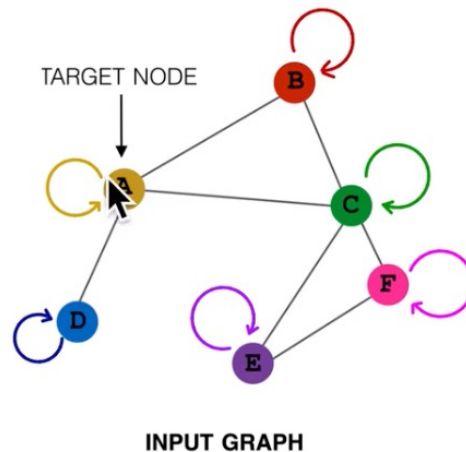
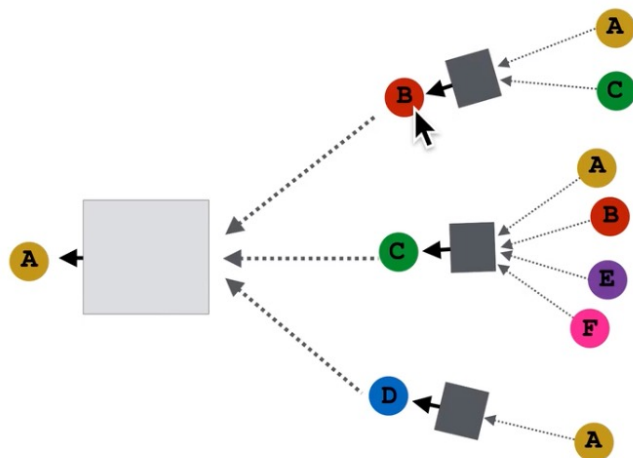
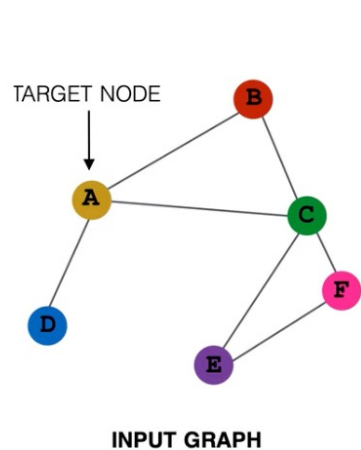
$$\sum_{u \in N(v)} \frac{h_u^k}{|N(v)|}$$

$D^{-1} A H^{(k)}$
Row Normalized Matrix
最大特征值为1

$$D^{-\frac{1}{2}} A D^{-\frac{1}{2}} H^{(k)}$$

Normalized Adjacency Matrix
Normalized Diffusion Matrix
Symmetric Normalized Matrix $\tilde{A}$

自环的引入 @ GCN



Initial 0-th layer embeddings are equal to node features

$$h_v^0 = x_v$$

本层节点v的嵌入向量

$$h_v^{(k+1)} = \sigma \left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)} \right), \forall k \in \{0, \dots, K-1\}$$

上-层节点v的嵌入向量 embedding of v at layer k

邻域节点上-层平均嵌入向量

Average of neighbor's previous layer embeddings

非线性激活函数 (e.g., ReLU)

非线性激活函数

Embedding after L layers of neighborhood aggregation

$$z_v = h_v^{(K)}$$

最后-层节点v的嵌入向量作为表示

Total number of layers

多个嵌入向量求和(平均)结果,与节点顺序无关
Notice summation is a permutation invariant pooling/aggregation.

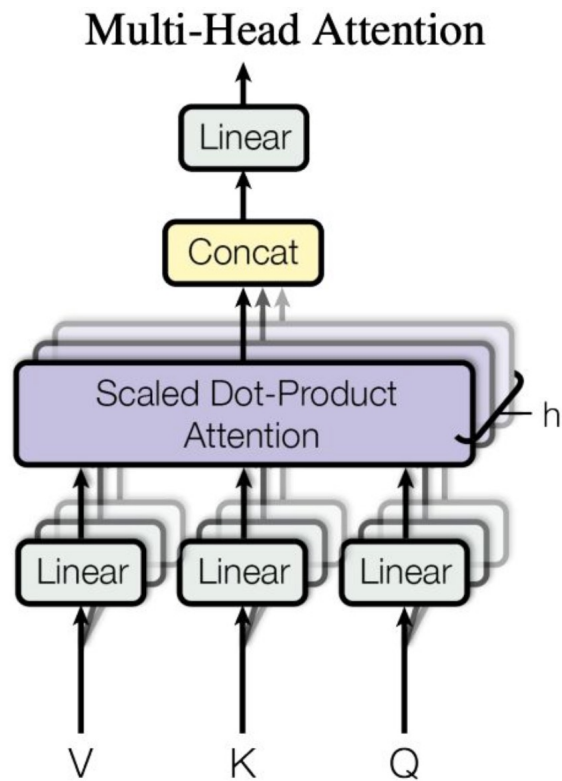


CNN 与 GNN

维度	卷积神经网络 (CNN)	图神经网络 (GNN)	差异点
数据结构	规则网格 (如图像中的二维网格)	任意图结构 (如社交网络、分子结构)	从规则网格拓展到任意拓扑结构
邻域定义	固定大小 (如 3x3 窗口)	动态邻域, 由图的连接关系决定	从固定邻域到动态邻域
权重共享	卷积核权重在整个输入图像中共享	根据边的关系可自适应调整权重 (如注意力机制)	从共享权重到自适应权重
感受野	局部感受野, 通过多层卷积逐渐扩展	多层邻域聚合直接扩大感受野	从局部感受野到可扩展的全局感受野
卷积操作	空间卷积, 基于像素的相对位置	图卷积, 基于节点的邻接关系	从空间卷积到图卷积
特征提取	局部特征提取, 识别边缘、纹理等	全局结构理解, 捕获节点间复杂关系	从局部特征提取到全局结构理解
输入数据类型	图像、视频、语音等规则网格数据	社交网络、分子、知识图谱等不规则数据	从欧氏空间数据拓展到非欧氏空间数据
信息聚合	卷积操作在固定邻域内滑动, 局部信息聚合	邻域信息聚合, 通过消息传递进行多跳邻域信息传递	从局部卷积滑动到多跳邻域信息聚合
权重调整方式	固定卷积核权重, 不考虑像素间差异	邻接矩阵权重可根据节点特征动态调整	引入注意力机制, 支持自适应权重分配
示例模型	LeNet、AlexNet、VGG、ResNet	GCN、GraphSAGE、GAT、MPNN	从经典CNN模型到GNN模型的应用
对平移不变性的适应	自然适应平移不变性, 卷积核共享实现平移不变性	使用权重共享的聚合函数实现节点间信息对称	从空间平移不变性到拓扑结构平移不变性
聚合策略	池化操作用于下采样, 减少计算	无需池化, 通过多层结构逐层捕获全局信息	从池化下采样到多层聚合

Transformer as a GNN

1. **Transformer 作为图神经网络**: Transformer 可以视为一种特殊的图神经网络, 因为它基于全连接的图结构, 并具有注意力机制。
 - **全连接图**: Transformer 模型中的每个节点 (即序列中的每个元素) 都与其他节点相连, 构成了全连接图。
 - **注意力机制**: Transformer 通过注意力机制来调整节点之间的连接权重, 使其在计算上类似于注意力型的图神经网络 (例如 GAT)。
2. **位置嵌入 (Positional Embeddings)**: 由于 Transformer 本质上是对称的, 因此需要引入位置嵌入以表示顺序信息。如果移除位置嵌入, 那么 Transformer 就等同于一个全连接的 GAT 模型。
3. **注意力作为软邻接关系**: 注意力机制可以理解作为一种软邻接关系, 不同节点之间的注意力权重可以看作是一种隐式的邻接矩阵。





从几何对称性角度理解深度学习网络

	卷积神经网络 (CNN)	图神经网络 (GNN)	循环神经网络 (RNN)	Transformer	DeepSets
几何对称性	平移不变性	置换不变性	时间平移不变性	序列位置不变性	置换不变性
适用数据	图像数据	图结构数据	时间序列数据	全局依赖的序列数据	集合数据
方法	卷积核在空间上局部移动，意味着即使输入图像发生平移，卷积层依旧能够检测相同的特征	通过图聚合操作（如平均、求和等）将邻居节点的特征聚合，GNN 实现了对节点置换的不变性。这种置换不变性确保了无论节点在何种顺序，模型的输出对特定节点都是一致的	在处理时间序列时，逐个时间步处理输入数据，隐藏状态传递信息并记忆之前的状态。RNN 能够捕捉序列中的时间平移特性，即相似模式无论在序列的不同位置出现都可以被识别	通过自注意力机制实现对序列中各元素的全局关联建模。Transformer 不依赖序列的顺序，而是通过位置编码来添加位置信息，使其对输入序列的置换仍然敏感。它通过捕获全局关系，打破了之前序列模型中的时间相关性限制	输入的元素顺序不影响模型的输出。通过对输入元素进行独立编码后再聚合（如加权求和、平均），DeepSets 可以对集合中元素的任意置换产生相同的输出



经典算法解读

SE(3)-Transformers: 3D Roto-Translation Equivariant Attention Networks

Fabian B. Fuchs^{*†}

Bosch Center for Artificial Intelligence
A2I Lab, Oxford University
fabian@robots.ox.ac.uk

Daniel E. Worrall^{*}

Amsterdam Machine Learning Lab, Philips Lab
University of Amsterdam
d.e.worrall@uva.nl

Volker Fischer

Bosch Center for Artificial Intelligence
volker.fischer@de.bosch.com

Max Welling

Amsterdam Machine Learning Lab
University of Amsterdam
m.welling@uva.nl

3D点云和图形的自注意力机制的变体，能够在3D旋转平移的过程中保持等变。



相对位置编码 @ (SE(3)-transformer)

定义： 相对位置编码用于表示输入点之间的相对距离和角度关系，而不是它们的绝对坐标。这是因为三维空间中的点的绝对位置并不重要，重要的是它们之间的相对位置。

实现：

- 对于点云中的每一对点 p_i 和 p_j ，计算它们的相对位置：

$$r_{ij} = p_j - p_i$$

- 使用相对位置向量 r_{ij} 来替代绝对坐标进行特征计算和注意力分配。

优点： 相对位置编码确保了模型对输入的旋转和平移具有鲁棒性，因为点的相对关系不会因整个结构的变换而改变。



球谐函数 @ (SE(3)-transformer)

定义： 球谐函数是一组在球面上定义的正交函数，广泛用于表示三维空间中的旋转不变特征。它们能够有效地捕捉方向性信息和几何特征。

实现：

- 在计算相对位置时，利用球谐函数将相对位置向量 r_{ij} 转换为球面坐标，提取其方向性信息。
- 球谐函数的应用可以将相对位置编码为旋转不变的特征。

优点： 通过使用球谐函数，模型能够学习到点之间的相对方向性，这对于理解三维结构的几何形状非常重要。



径向神经网络 @ (SE(3)-transformer)

定义：径向神经网络（Radial Neural Networks, RNN）是一种专门设计用于处理输入点之间距离的神经网络。它通过对距离进行学习，能够有效地捕捉非线性关系。

实现：

- 将相对位置向量的模长（即距离）作为输入，通过径向神经网络进行处理。RNN可以学习到距离和特征之间的复杂关系。
- 结合球谐函数提取的方向信息和相对位置的模长，创建一个完整的输入特征表示。

优点：径向神经网络能够捕捉到距离对特征的重要性，同时与方向性信息结合，从而增强模型的表达能力。



注意力的实现 @ (SE(3)-transformer)

输入特征：为每个点 p_i 提取特征 h_i 并计算其与其他点 p_j 的相对位置编码 r_{ij} 。

特征融合：使用球谐函数将相对位置编码转换为方向性特征，结合径向神经网络生成新的特征表示：

$$\text{feature}(p_i, p_j) = \text{RNN}(\|r_{ij}\|) \oplus \text{Spherical Harmonics}(r_{ij})$$

其中 $\oplus$ 表示特征的拼接或融合。

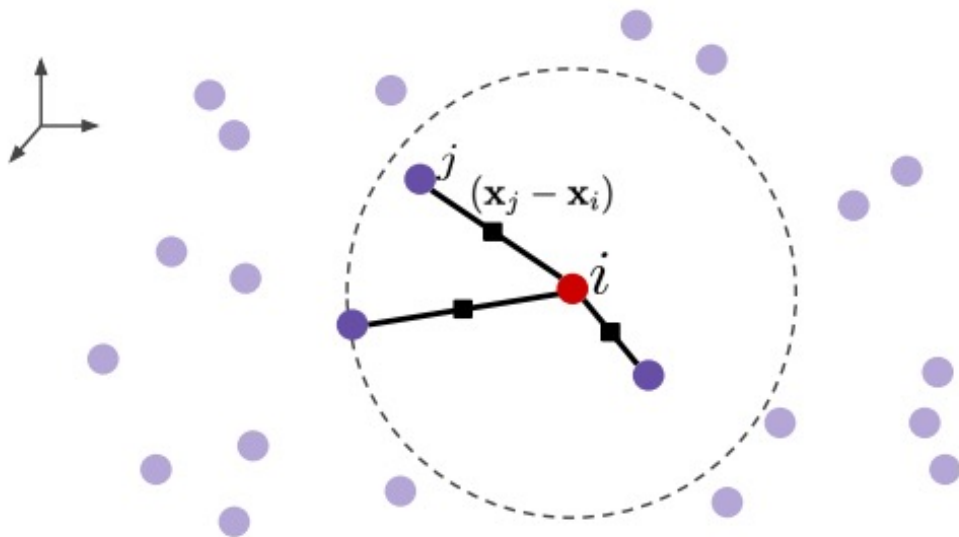
注意力权重计算：利用上述生成的特征进行自注意力计算，确保在计算注意力权重时考虑了点之间的几何关系：

$$a_{ij} = \frac{\exp(f(h_i, \text{feature}(p_i, p_j)))}{\sum_k \exp(f(h_i, \text{feature}(p_i, p_k)))}$$

特征更新：根据注意力权重更新每个点的特征，实现基于几何关系的特征聚合。

Step – 1 @ (SE(3)-transformer)

Step 1: Get nearest neighbours and relative positions



Step 1: 获取邻居和相对位置

- 首先确定目标节点 i 的邻居节点 j 以及它们的相对位置 $x_j - x_i$ 。
- 这种局部邻域关系可以在三维空间中确定每个节点与其邻居之间的距离和方向。
- 通过相对位置编码确保网络对旋转和平移操作的鲁棒性。
- 引入三维几何信息，使得网络能够有效建模节点间的局部关系。
- 降低计算复杂度，保证模型在处理大规模三维数据时的效率。

为等变性提供基础：

- SE(3) 等变网络的目标是保持网络在三维旋转和平移下的等变性。通过使用相对位置向量 r_{ij} 而不是绝对位置，可以确保网络只依赖于节点之间的相对空间关系，而不受整体旋转或平移的影响。
- 这种相对位置表示是等变特征构建的基础，因为相对位置在旋转或平移操作下会发生相应的变化，但其相对位置的方向和距离关系保持不变，从而实现了等变性。

引入三维几何关系：

- 通过相对位置向量 r_{ij} ，网络能够捕捉到节点之间的距离（通过 $\|r_{ij}\|$ ）和方向信息（通过单位向量 $\frac{r_{ij}}{\|r_{ij}\|}$ ）。
- 距离信息在后续步骤中被输入到径向神经网络中，用于捕捉非线性关系，而方向信息则通过球谐函数编码捕捉旋转不变特征。
- 这两部分信息结合使得网络能够更好地理解节点局部几何结构，有利于三维数据的准确建模。

减少计算复杂度：

- 仅选择最近邻居可以显著减少计算复杂度，避免在大规模数据中进行全局计算。尤其在点云、分子建模等应用中，每个节点仅与其局部域内的节点进行交互，大大降低了计算成本。
- 这种局部交互也符合物理规律，例如分子结构中，原子间的相互作用通常只依赖于其最近邻居的作用力。

Step – 2 @ (SE(3)-transformer)

Step 2: Get SO(3)-equivariant weight matrices



Clebsch-Gordon Coeff.

$$Q_{Jm}^{\ell k}$$



Radial Neural Network

$$\varphi_J^{\ell k}(\|x\|)$$



Spherical Harmonics

$$Y_{Jm}\left(\frac{x}{\|x\|}\right)$$

- **Clebsch-Gordon系数** ($Q_{Jm}^{\ell k}$): 这是量子力学中的一个概念，用于在不同角动量之间转换。在SE(3)网络中，它被用来帮助特征在不同角度之间的映射和组合，确保旋转等变性。
- **作用**: 这些系数使得不同旋转角度和级别的特征能够在权重矩阵中正确组合，从而保持SO(3)等变性。

- **径向神经网络** ($\varphi_J^{\ell k}(\|x\|)$): 这是一个针对距离或模长 ($\|x\|$) 的神经网络，用于处理点之间的距离信息。径向神经网络捕捉到在空间中的距离特征，并将其映射为网络的权重。
- **作用**: 通过对距离的非线性处理，径向神经网络使模型能够学习到与点之间距离相关的特征，从而补充球谐函数的方向性信息。

- **球谐函数** ($Y_{Jm}\left(\frac{x}{\|x\|}\right)$): 球谐函数是一种在球面上定义的正交函数，主要用于提取三维方向信息。在这里，它根据输入点的方向（单位向量 $\frac{x}{\|x\|}$ ）提取旋转不变特征。
- **作用**: 球谐函数能够将点的方向信息编码到模型中，使得注意力机制可以对方向敏感。这在保持旋转等变性方面非常重要。

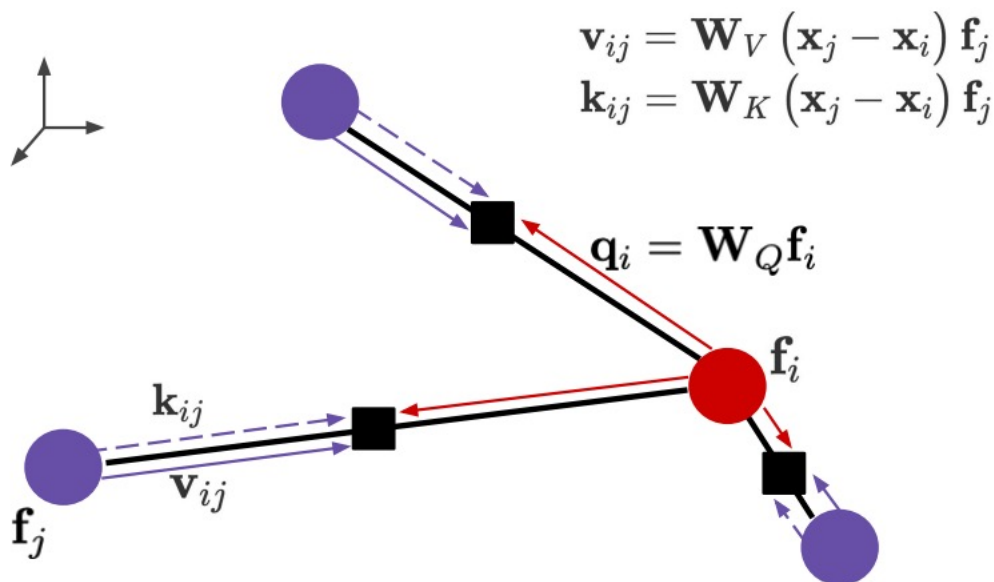
Matrix W consists of blocks mapping between degrees

$$W(x) = W \left(\left\{ Q_{Jm}^{\ell k}, \varphi_J^{\ell k}(\|x\|), Y_{Jm}\left(\frac{x}{\|x\|}\right) \right\}_{J,m,\ell,k} \right)$$

- **组合**: 权重矩阵 $W(x)$ 是通过三个组件的组合构成的。具体来说，它由Clebsch-Gordon系数 $Q_{Jm}^{\ell k}$ 、径向神经网络输出 $\varphi_J^{\ell k}(\|x\|)$ 和球谐函数 $Y_{Jm}\left(\frac{x}{\|x\|}\right)$ 构成。
- **结构**: 矩阵 W 包含不同角度和距离的块映射 (blocks mapping)，从而在三维空间中的不同“度”之间建立了映射关系，确保输出能够保持与输入旋转和方向的对应关系。

Step – 3 @ (SE(3)-transformer)

Step 3: Propagate queries, keys, and values to edges



Step 3: 将查询、键和值传播到边

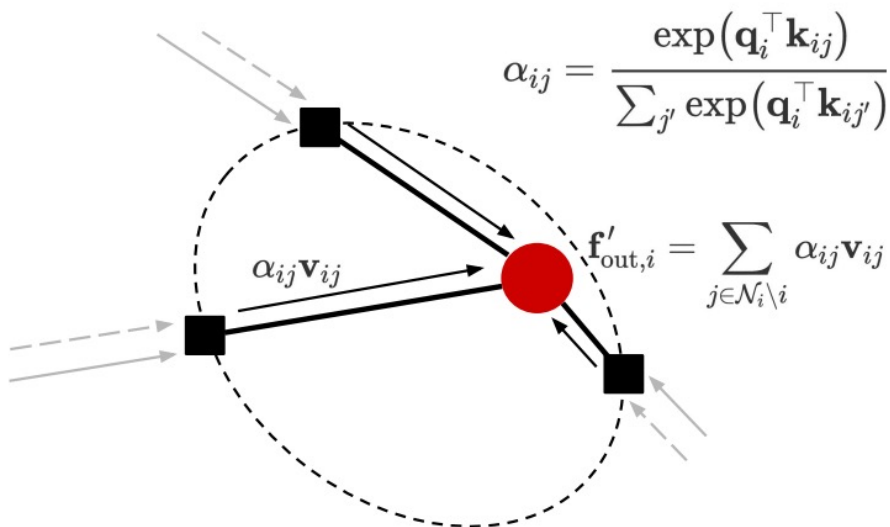
- 对每个节点 i 生成查询向量 q_i ，并对每个边生成键 k_{ij} 和值 v_{ij} 向量，具体计算为：
 - $v_{ij} = W_V(x_j - x_i)f_j$
 - $k_{ij} = W_K(x_j - x_i)f_j$
 - $q_i = W_Q f_i$
- 这些操作通过相对位置和特征计算，使得注意力机制能够适应几何图结构。

相对位置编码和等变性保障：

- 在生成键向量和值向量时，通过引入相对位置 $(x_j - x_i)$ ，确保特征向量能够感知三维空间中的相对几何关系。这种方式结合了**Step 2**中相对位置编码的思想，通过将相对位置和节点特征共同输入，保证了等变性。
- 使用的权重矩阵 W_K 和 W_V 以及球谐函数、径向神经网络来对相对位置向量进行编码，确保即使在三维空间中的旋转和平移下，生成的键和值依然符合等变性要求。

Step – 4 @ (SE(3)-transformer)

Step 4: Compute attention and aggregate



1. 注意力权重的计算:

- 每个节点 i 与邻居节点 j 之间的注意力权重 α_{ij} 通过查询向量 $\mathbf{q}_i$ 和键向量 $\mathbf{k}_{ij}$ 的内积计算得到:

$$\alpha_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_{ij})}{\sum_{j'} \exp(\mathbf{q}_i^\top \mathbf{k}_{ij'})}$$

- 查询向量 $\mathbf{q}_i$ 和键向量 $\mathbf{k}_{ij}$ 是在 Step 3 中生成的:

- $\mathbf{q}_i = W_Q f_i$, 即从节点特征 f_i 经过线性变换得到。
- $\mathbf{k}_{ij} = W_K(x_j - x_i)f_j$, 结合节点 j 的特征 f_j 和相对位置 $(x_j - x_i)$ 生成。

注意力计算的等变性:

- 在计算注意力权重 α_{ij} 时, 使用了查询向量和键向量的内积。由于这些向量中包含了等变的相对位置编码, 旋转和平移变换不会改变相对位置, 因此注意力权重对旋转和位移操作保持一致性。
- 这样, 注意力权重 α_{ij} 对输入的旋转和位移变换保持不变或相应变化, 确保了等变性。

2. 特征聚合:

- 使用注意力权重 α_{ij} 对值向量 $\mathbf{v}_{ij}$ 进行加权:

$$\mathbf{f}'_{\text{out},i} = \sum_{j \in \mathcal{N}_i \setminus i} \alpha_{ij} \mathbf{v}_{ij}$$

- 值向量 $\mathbf{v}_{ij} = W_V(x_j - x_i)f_j$, 包含节点 j 的特征 f_j 和相对位置的几何信息。

通过这个加权求和, 节点 i 的新特征 $\mathbf{f}'_{\text{out},i}$ 得到更新, 综合了邻居节点的信息。

特征聚合的等变性:

- 在特征聚合过程中, 节点 i 的特征更新 $\mathbf{f}'_{\text{out},i}$ 是邻居节点值向量的加权和, 这些值向量中包含了等变的相对位置和特征信息。
- 因此, 当输入发生旋转和平移时, 聚合后的特征更新也会相应地发生变化, 保持整个网络的等变性。



在几何图上的学习方法 (SE(3)-transformer)

SE(3)-Transformers 是一种将 Transformer 机制引入三维数据处理的架构。在标准 Transformer 的基础上，SE(3)-Transformers 通过引入几何注意力机制，使得模型能够在三维空间中捕捉不同点之间的几何关系，从而对输入进行等变处理。

SE(3)-Transformer通过相对位置编码、球谐函数和径向神经网络的组合，实现了在三维几何结构下的注意力计算，并确保了旋转和平移等变性。这种设计适合分子建模等需要对几何关系保持不变的任务。

1. 邻域限制：SE(3)-Transformer的操作仅作用于节点的邻域 $\mathcal{N}_i$ ，这可以减少计算量并聚焦于局部信息传递。

3. 注意力权重计算：

- 注意力权重 α_{ij} 的计算方式为：

$$\alpha_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_{ij})}{\sum_{j' \in \mathcal{N}_i \setminus \{i\}} \exp(\mathbf{q}_i^\top \mathbf{k}_{ij'})}$$

- 其中， $\mathbf{q}_i$ 和 $\mathbf{k}_{ij}$ 分别是查询向量和键向量，基于节点和邻居的信息生成。

2. 输出特征更新公式：

- 节点 i 的输出特征 $\mathbf{f}_{\text{out},i}^\ell$ 由三部分组成：

$$\mathbf{f}_{\text{out},i}^\ell = \underbrace{\mathbf{W}_V^{\ell\ell} \mathbf{f}_{\text{in},i}^\ell}_{\text{自交互}} + \sum_{k \geq 0} \sum_{j \in \mathcal{N}_i \setminus \{i\}} \underbrace{\alpha_{ij}}_{\text{注意力权重}} \underbrace{\mathbf{W}_V^{\ell k} (\mathbf{x}_j - \mathbf{x}_i) \mathbf{f}_{\text{in},j}^k}_{\text{数值消息}}$$

- 其中：

- 自交互： $\mathbf{W}_V^{\ell\ell} \mathbf{f}_{\text{in},i}^\ell$ 表示节点自身特征的线性变换。
- 注意力权重 α_{ij} ：控制邻居节点 j 对节点 i 的影响力，类似于 Transformer 中的注意力机制。
- 数值消息： $\mathbf{W}_V^{\ell k} (\mathbf{x}_j - \mathbf{x}_i) \mathbf{f}_{\text{in},j}^k$ 是邻居节点的信息，根据相对坐标 $(\mathbf{x}_j - \mathbf{x}_i)$ 进行加权。

4. 查询和键的计算：

- 查询向量 $\mathbf{q}_i$ 和键向量 $\mathbf{k}_{ij}$ 的计算如下：

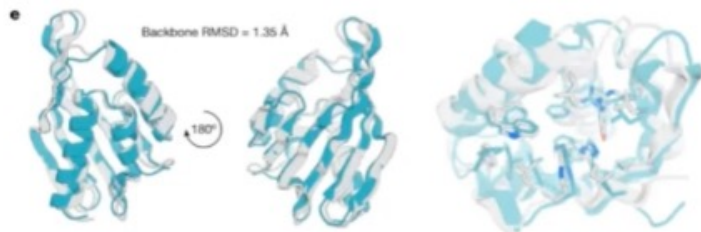
$$\mathbf{q}_i = \bigoplus_{\ell \geq 0} \sum_{k \geq 0} \mathbf{W}_Q^{\ell k} \mathbf{f}_{\text{in},i}^k$$

$$\mathbf{k}_{ij} = \bigoplus_{\ell \geq 0} \sum_{k \geq 0} \mathbf{W}_K^{\ell k} (\mathbf{x}_j - \mathbf{x}_i) \mathbf{f}_{\text{in},j}^k$$

- 查询和键的定义使得注意力机制能够适应于几何图的相对位置关系。



Rosetta(1998): 基于能量函数的
蛋白质结构预测与设计
<https://www.rosettacommons.org/about/pubs>



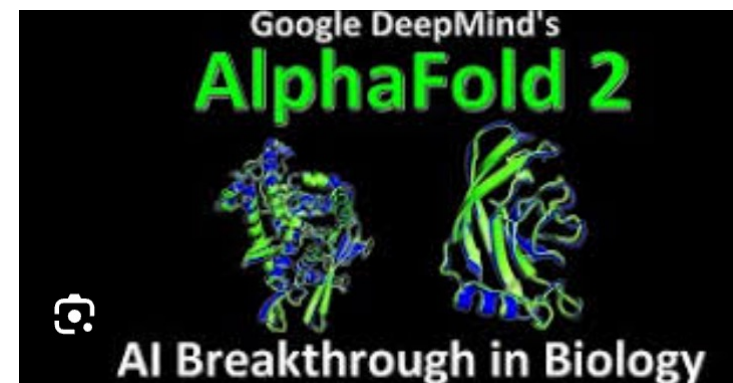
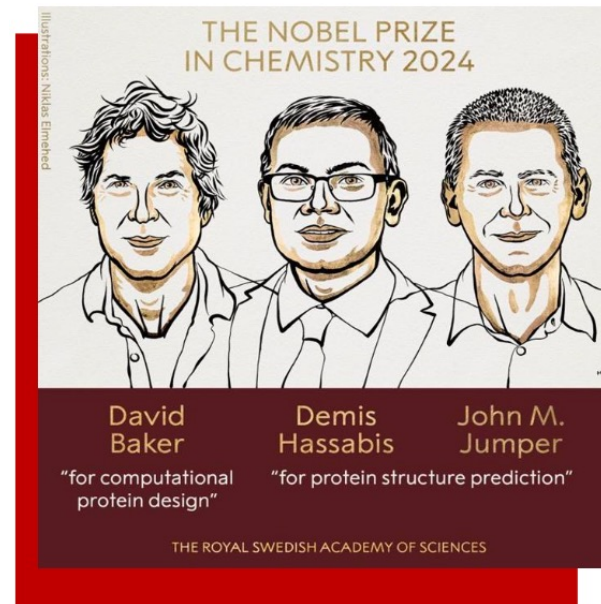
荧光素酶的设计: 人类第一个从头设计的酶
(2023)

Yeh, A.H.W., Norn, C., Kipnis, Y. *et al.*
Nature **614**, 774–780 (2023).

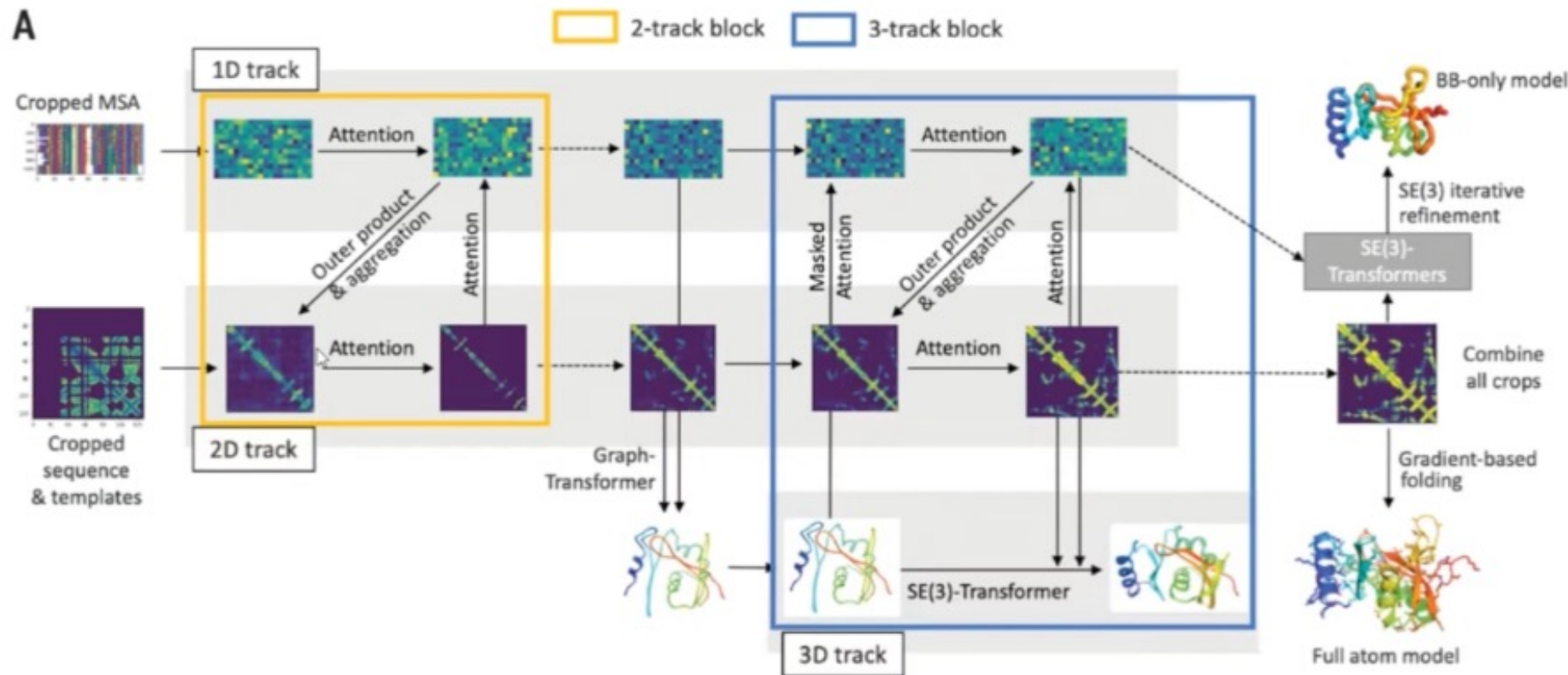
▲ 41



RosettaFold(2021)
深度学习预测蛋白质结构



几何等变性 @ rosettafold



迭代细化： 在 3D 轨道中，图的节点和边构成蛋白质的原子和键，模型通过 SE(3)-等变性模块进行细化，每次迭代都在等变性下更新几何关系，从而确保模型能够捕获全原子结构。

梯度折叠： 最后，通过梯度折叠过程将 3D 信息结合，得到蛋白质的全原子模型，该过程结合了等变性约束，从而生成具有物理合理性的结构

三轨道结构： 该框架分为 1D、2D 和 3D 三个轨道模块，这些模块通过注意力机制相互作用。例如，1D 和 2D 轨道通过外积和聚合进行交互。这种设计使得信息在不同维度中流动，从而保持了等变性，因为不同维度的信息可以统一到 3D 空间中。

SE(3)-等变性模块： 在 3D 轨道中使用了 SE(3)-等变性变换器 (SE(3)-Transformers)，这类变换器在空间变换（包括旋转和平移）下保持不变性。由于蛋白质的三维结构不应该因为坐标系的旋转或移动而改变，这种等变性是必要的，能够使模型更好地保留蛋白质结构的几何信息

几何等变性 @ alphafold2

单体和成对表示：

- 模型利用了两种表示：单体表示 (r,c) 和 成对表示 (r,r,c) 。
- 单体表示用于编码单个残基的信息，而成对表示用于编码残基之间的成对关系信息，这些表示是蛋白质结构预测的基础。

IPA模块：

不变点注意力（IPA）模块：实现几何等变的关键。这意味着它可以在不丢失关系信息的情况下处理输入结构的旋转和平移，这是捕捉蛋白质3D构型的关键。

预测相对旋转和平移：

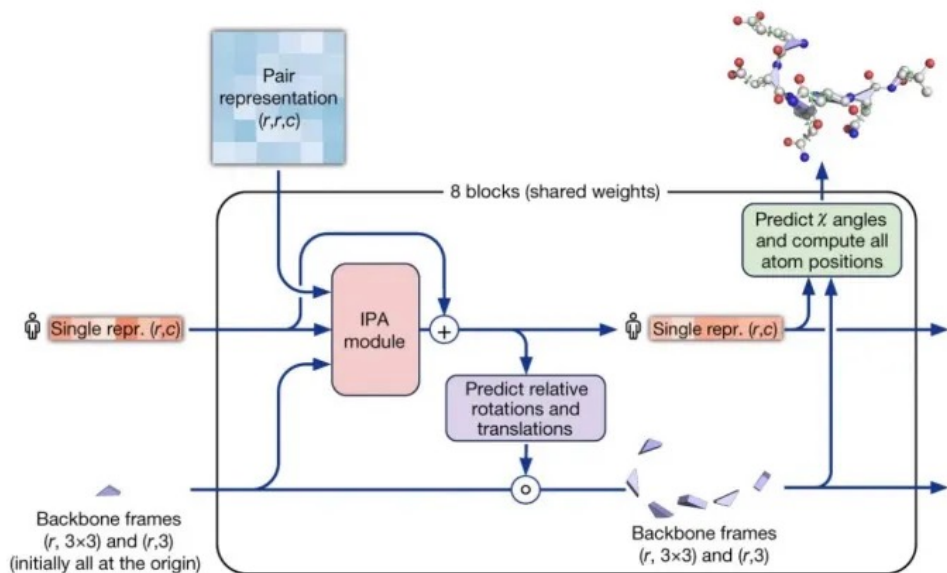
•IPA模块输出残基之间的相对旋转和平移，这些信息被用于更新蛋白质的主链框架，从而定义其空间结构。

主链框架和角度预测：

- 主链框架 $(r, 3 \times 3)$ 和 $(r, 3)$ 最初都初始化在在原点，随后在模型中逐步更新。这些框架表示残基在3D空间中的方位和位置。
- 经过多层处理（图中共有8层，并共享权重），模型最终预测出角度（ χ 角），它们定义了侧链的精确取向。利用这些角度，模型计算出蛋白质中所有原子的3D位置。

迭代过程：

- 模型通过8个共享权重的块不断迭代这个过程，每一层逐步更新主链框架、单体和成对表示，从而逐步精确化3D结构的预测。
- 每一层都会对结构进行更新和精炼，使得预测结果越来越接近真实的蛋白质折叠结构。



AlphaFold 2 brought a breakthrough in predicting protein folding structure. At the core of this architecture is the “invariant point attention” module implementing geometric equivariance. Image from Jumper et al. [95].



Do Transformers Really Perform Badly for Graph Representation?

Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, [Tie-Yan Liu](#)
[NeurIPS 2021](#) | December 2021

[↓ Download BibTex](#)

The Transformer architecture has become a dominant choice in many domains, such as natural language processing and computer vision. Yet, it has not achieved competitive performance on popular leaderboards of graph-level prediction compared to mainstream GNN variants. Therefore, it remains a mystery how Transformers could perform well for graph representation learning. In this paper, we solve this mystery by presenting Graphormer, which is built upon the standard Transformer architecture, and could attain excellent results on a broad range of graph representation learning tasks, especially on the recent OGB Large-Scale Challenge. Our key insight to utilizing Transformer in the graph is the necessity of effectively encoding the structural information of a graph into the model. To this end, we propose several simple yet effective structural encoding methods to help Graphormer better model graph-structured data. Besides, we mathematically characterize the expressive power of Graphormer and exhibit that with our ways of encoding the structural information of graphs, many popular GNN variants could be covered as the special cases of Graphormer.

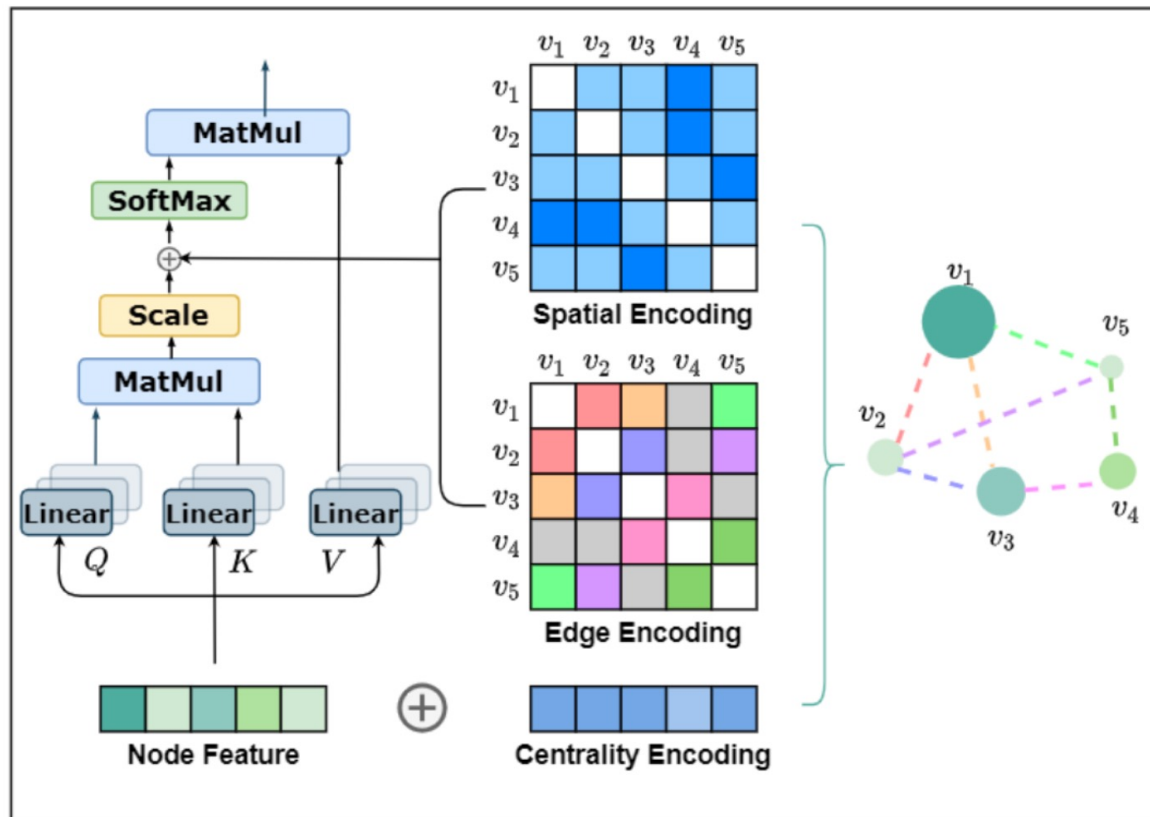
GRAPHORMER

= Transformer

+ 空间编码

+ 中心性编码

+ 连边编码





MPNN

消息传递机制 (Message Passing) 引入量子化学和分子性质预测领域

消息传递机制
(Message Passing Mechanism) :

$$m_i^{(t+1)} = \sum_{j \in \mathcal{N}(i)} M_t(h_i^{(t)}, h_j^{(t)}, e_{ij})$$

其中:

- $m_i^{(t+1)}$ 是节点 i 在第 $t+1$ 层的接收消息;
- $h_i^{(t)}$ 是节点 i 在第 t 层的特征表示;
- e_{ij} 是节点 i 和节点 j 之间的边特征 (例如化学键类型);
- $\mathcal{N}(i)$ 表示节点 i 的邻域;
- M_t 是一个可学习的消息传递函数, 通常是多层感知器 (MLP) 或其他非线性变换。

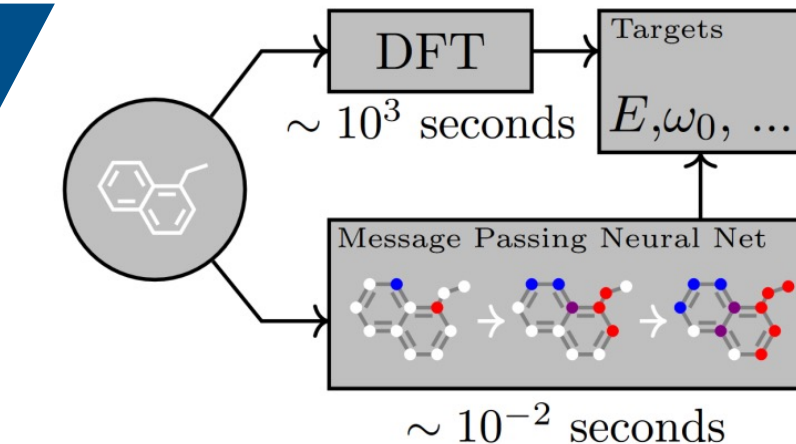
特征聚合与更新
(Feature
Aggregation and
Update)

$$h_i^{(t+1)} = U_t(h_i^{(t)}, m_i^{(t+1)})$$

$h_i^{(t+1)}$ 是节点 i 在第 $t+1$ 层的特征表示;

U_t 是一个更新函数, 通常是 RNN (如 GRU) 或其他非线性激活函数;

$m_i^{(t+1)}$ 是在前一步计算得到的节点 i 的聚合消息。



全局池化 (Global Pooling)

$$\hat{y} = R(\{h_i^{(T)} | i \in V\})$$

其中:

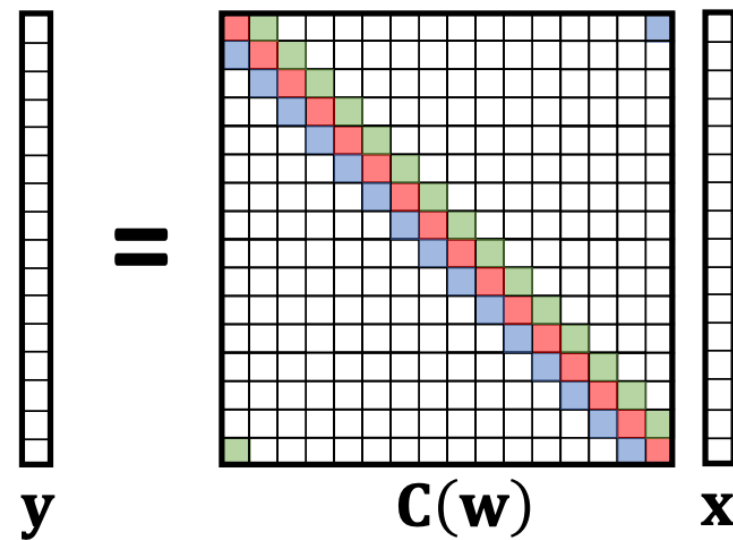
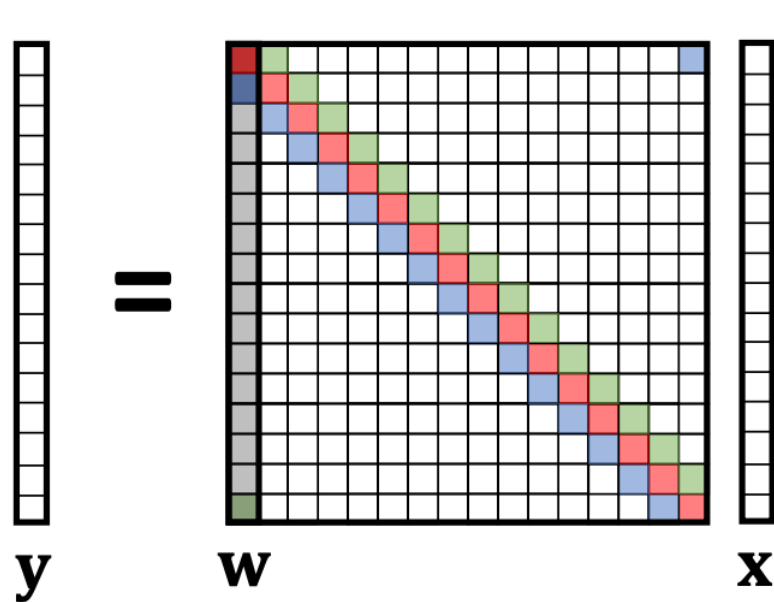
- $\hat{y}$ 是整个图的全局表示, 可以用来进行预测;
- R 是一个聚合函数 (如求和、平均或最大值池化), 将图中所有节点的最终特征 $h_i^{(T)}$ 聚合成一个全局向量;
- V 表示图中的所有节点。

Neural Message Passing for Quantum Chemistry

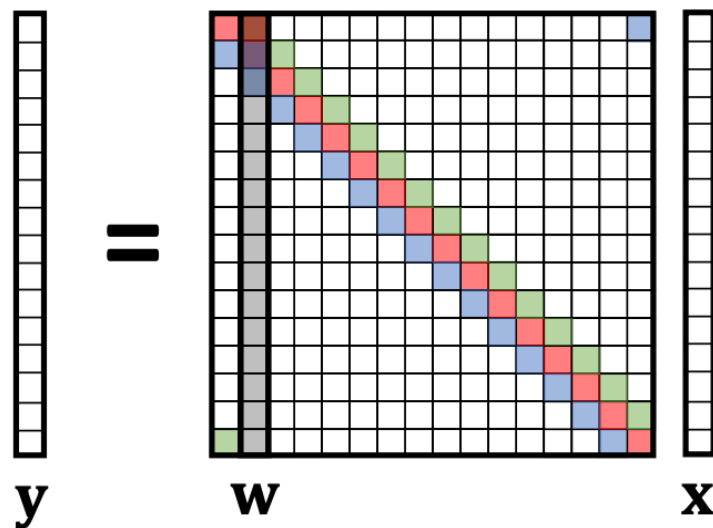
Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, George E. Dahl, ICML2017



附录



Convolution = circulant matrix





沈括 (1031年–1095年)

秦九韶 (1208年–1261年)

张邱建 (约5世纪)

朱世杰 (1249年–1314年)

祖冲之 (429年–500年)

刘徽 (约225年–295年)

杨辉 (约1238年–1298年)

spatial domain

Circulant matrix

$\mathbf{C}(\mathbf{w})$

$\mathbf{x}$

$\mathbf{w} \star \mathbf{x}$

Φ Φ^* DFT

IDFT Φ Φ^*

$\hat{\mathbf{x}}$

$\widehat{\mathbf{w} \star \mathbf{x}}$

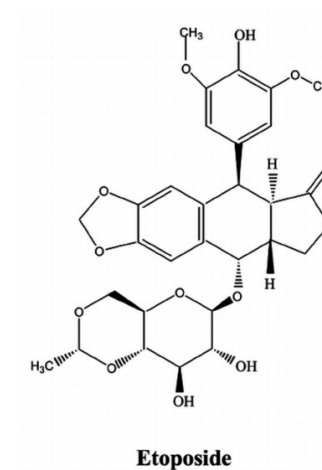
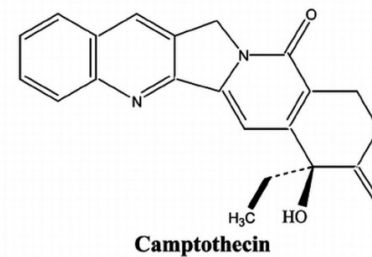
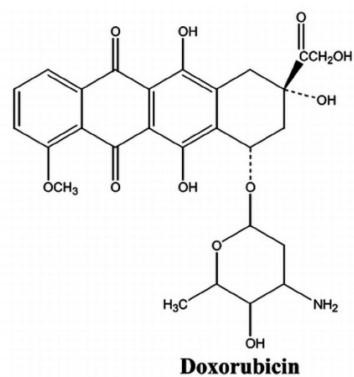
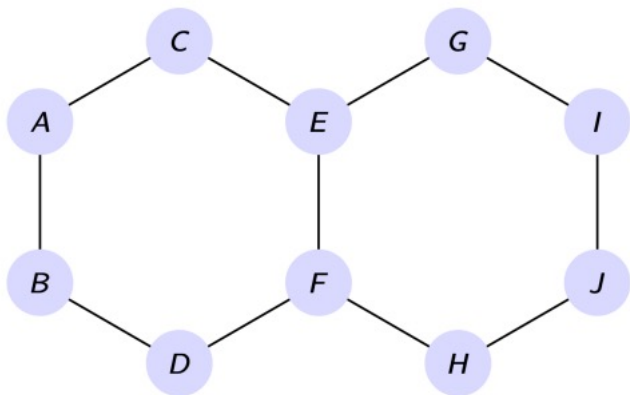
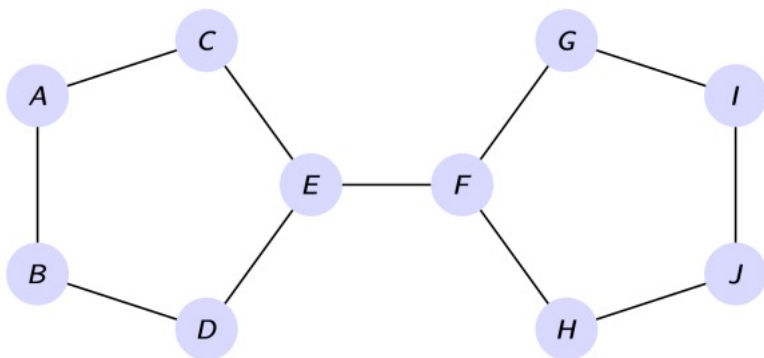
$\begin{bmatrix} \hat{w}_1 & & \\ & \ddots & \\ & & \hat{w}_n \end{bmatrix}$

Element-wise product

frequency domain

- **Convolution in spatial domain:** circulant matrix
- **Local aggregation on adjacent nodes** with shared parameters
- Special structure due to underlying grid
- All circulant matrices diagonalized by DFT (eigenvectors of shift)
- **Convolution in frequency domain:** apply DFT, apply element-wise product, apply inverse DFT
- Efficient computation: $O(n)$ with small filter
 $O(n \log n)$ using FFT

The bonds between different atoms in chemical compounds form cycles. The hexagon is the most common polygon in organic compounds $\Rightarrow$ yields stable materials.





空间域中的卷积



空间域中的卷积

循环矩阵 (Circulant Matrix)

每一行或每一列元素都是前一行或前一列元素通过循环移位得到的

$$C = \begin{bmatrix} c_0 & c_1 & c_2 & \cdots & c_{n-1} \\ c_{n-1} & c_0 & c_1 & \cdots & c_{n-2} \\ c_{n-2} & c_{n-1} & c_0 & \cdots & c_{n-3} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_1 & c_2 & c_3 & \cdots & c_0 \end{bmatrix}$$

循环移位结构

可对角化

矩阵乘法的高效性

Two sides of the same coin

Generative modeling



Known: samples

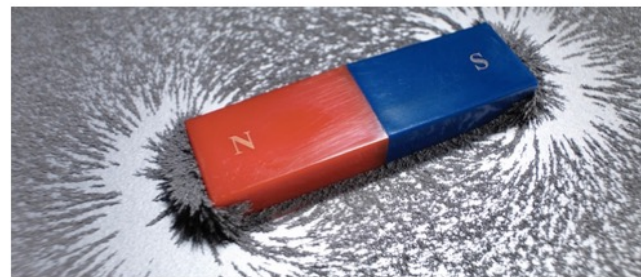
Unknown: generating distribution

“learn from data”

$$\mathcal{L} = - \mathbb{E}_{x \sim \text{data}} [\ln p(x)]$$

$$\mathbb{KL}(\text{data} \parallel p) \text{ vs } \mathbb{KL}(p \parallel e^{-E/k_B T})$$

Statistical physics



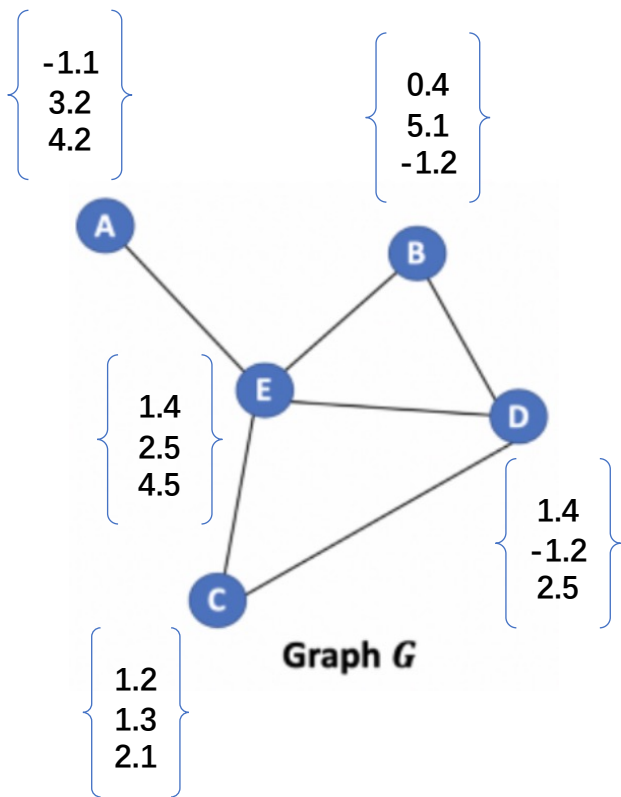
Known: energy function

Unknown: samples, partition function

“learn from energy”

$$F = \mathbb{E}_{x \sim p(x)} [E(x) + k_B T \ln p(x)]$$

$$G = \langle V, E \rangle$$



	A	B	C	D	E
A	0	0	0	0	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	1	1	0	1
E	1	1	1	1	0

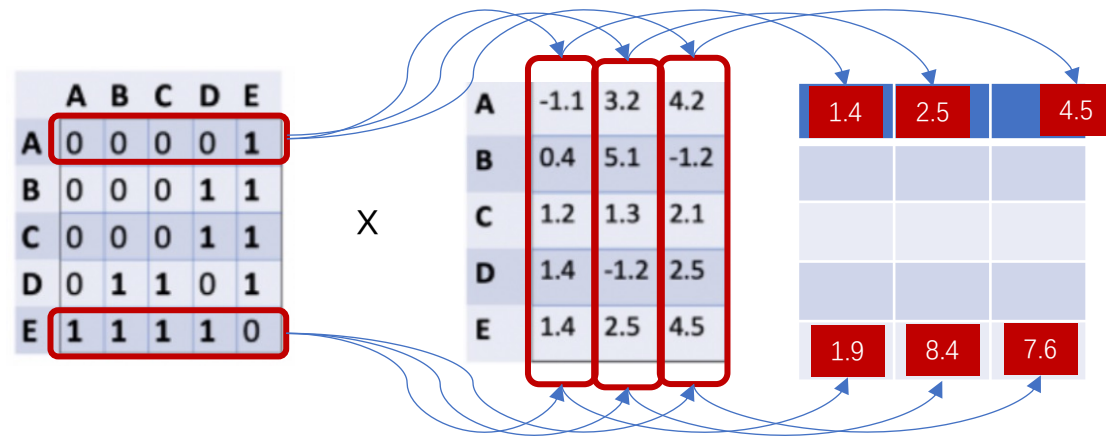
Adjacency matrix

	A	B	C	D	E
A	1	0	0	0	0
B	0	2	0	0	0
C	0	0	2	0	0
D	0	0	0	3	0
E	0	0	0	0	4

degree matrix

	A	B	C
A	-1.1	3.2	4.2
B	0.4	5.1	-1.2
C	1.2	1.3	2.1
D	1.4	-1.2	2.5
E	1.4	2.5	4.5

Node vector



$$v_1 = Av_0$$

$$v_2 = Av_1 = A(Av_0) = A^2v_0$$

...

$$v_n = A^n v_0$$